LLM ENGINEERING

Week 04

Modern Attention

Thursday, 5 November 2026

Interactive version: https://llm.strueber.dev/#/week-04

Bethgelab · University of Tübingen · Winter term 2026/27

Text and figures under CC BY 4.0, code under the MIT licence.

Recap & Roadmap

Announcements

Next week, 12 November

- Week 5: Optimization

Quiz 2 on ILIAS

- Weeks 3, 4 and Bonus 2
- Due Friday 13 November, end of day

Third exercise: this lecture

- Due Wednesday 11 November, 22:00
- Email when it reaches your repository

▼ Dates for the coming week

Next week's lecture, on 12 November, is Week 5, on optimization.

Quiz 2 is on ILIAS and closes at the end of Friday, 13 November. It covers Week 3, Week 4 and Bonus 2.

The third coding exercise covers the content of this lecture and is due on Wednesday, 11 November, at 22:00. You will get an email when it is in your pair's repository.

How can more FLOPs run faster?

FlashAttention does 13% more arithmetic.
It runs three times faster.

What limits attention on a GPU, and how do models cut it?

- Roofline: is compute or memory traffic the limit?
- **FlashAttention**: exact attention, yet faster
- Sliding-window attention (SWA): recent tokens only
- **Multi-head latent attention** (MLA): less per token
- Exercise: build SWA and MLA, measure them

▼ Today's question

Start with a measurement that arithmetic alone does not explain. Dao et al. timed the attention of GPT-2 medium on an A100 GPU, forward and backward pass together, at a sequence length of 1,024, with the standard implementation and with their FlashAttention (Dao et al., 2022) . FlashAttention computes exact attention: in exact arithmetic its output equals the standard one.

Reported. In Figure 2 of Dao et al., standard attention performs 66.6 GFLOP in 35.1 ms, and FlashAttention performs 75.2 GFLOP in 11.7 ms (Dao et al., 2022). The ratios of floating-point operations (FLOPs) and of running times are

$$\begin{aligned} \text{FLOPs:} \quad & \frac{75.2}{66.6} \approx 1.13, \\ \text{time:} \quad & \frac{35.1}{11.7} = 3.0. \end{aligned}$$

So FlashAttention does 13% more arithmetic and runs three times faster. If the FLOP count set the running time, more operations could not take less time. The FlashAttention tab shows what sets it instead.

The question for this lecture is: what limits attention on a GPU, arithmetic or bytes, and how do modern models cut that cost? The answer has four parts.

1. The roofline compares the arithmetic an operation performs with the bytes it moves, and says which of the two limits it on a given device.
2. FlashAttention computes the same exact attention faster.
3. Sliding-window attention (SWA) lets each query read only the most recent tokens.
4. Multi-head latent attention (MLA) stores fewer numbers per token in the KV cache.

Along the way come attention sinks and two learned sparse-attention methods, DeepSeek sparse attention (DSA) and MiniMax sparse attention (MSA).

The exercise builds the sliding window and MLA and measures attention against the roofline. Nothing trains this week: every measurement is a forward pass.

Where Week 3 left off

- GQA: query heads share KV heads
- Masks decide which keys count
- RoPE rotates queries and keys, NoPE adds no position
- KV cache: keys and values of every past token and layer

Position and locality: separate choices

▼ What you bring from Week 3

Grouped-query attention

In grouped-query attention (GQA), H query heads share H_{kv} key-value (KV) heads, where H_{kv} divides H. Each KV head serves a group of $g = H/H_{kv}$ consecutive query heads. The model keeps all its query heads and stores fewer keys and values. Multi-head attention (MHA) is the case $H_{kv} = H$.

Masks and depth

A mask states which keys are visible to each query, and so it fixes what attention means, separately from how a kernel computes it. Week 3 composed one boolean mask. A query-key pair is allowed when both are real tokens, the key does not lie in the future, and both belong to the same document.

Week 3 also repeated the block into a stack of L layers. Each layer's mask sets what that layer can reach, and depth composes these reaches: what one layer gathers into a position, the next layer can pass on.

Position

From Week 3 on, the course model handles position with RoPE by default, and NoPE is its ablation and design option. Rotary position embedding (RoPE) rotates all coordinates of each query and each key, one pair at a time, by angles proportional to the token's position. Each rotary score term then depends on the two positions only through their offset. For one coordinate pair, q from the query at position m and k from the key at position n, rotated by $R_m = R(m\omega)$ and $R_n = R(n\omega)$ at the frequency ω, Week 3 derived the relative-score identity

$$(R_m q)^\top (R_n k) = q^\top R_{n-m}\, k.$$

A model with no positional encoding (NoPE) adds no position signal at all. Its causal mask still gives each query row a different visible prefix.

The KV cache

When a model generates text, the query of each new token attends to the keys and values of every earlier token, in every layer. The model stores them, for every sequence in the batch, instead of computing them again. Take S cached tokens per sequence and a model of L layers, each with H_{kv} KV heads of width D_h. Each sequence then occupies

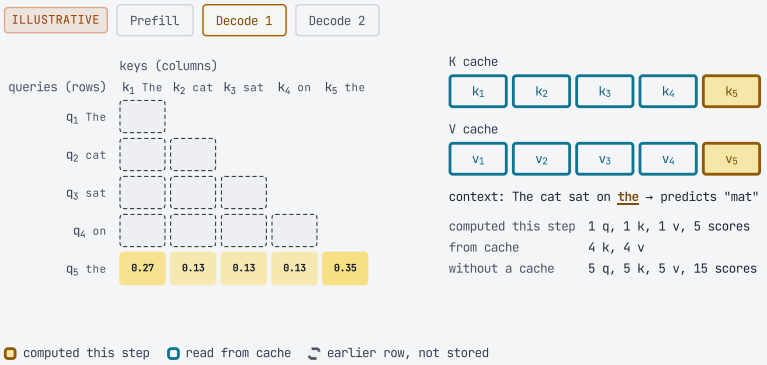
$$2\, S\, L\, H_{kv}\, D_h \text{ elements},$$

where the 2 counts keys and values, and a batch multiplies this by its number of sequences. In the Week 3 example, serving 32 sequences of 1,024 tokens, the cache took about ten times the memory of the weights. GQA shrinks it by reducing H_{kv}. This lecture asks what else limits attention and how else the stored state can shrink.

Position and locality

Position handling (RoPE or NoPE) changes how position affects attention scores. Locality (the causal sliding-window mask) changes which keys a query may attend to. They are separate design choices. A sliding-window mask lets each query read only the most recent keys. It combines with RoPE or with NoPE, and a model with either can also read its whole prefix. You do not need Bonus Chapter 2, on extending position encodings to long contexts, for this lecture.

Decode reads every cached key and value



Rows are queries and columns are keys, on the toy sentence from Week 2's heatmap. A filled cell is computed in the current step, and its number and shade give its attention weight. An empty cell above the diagonal is masked. The two strips list the key and the value of every position the model has seen.

- Decode: one new query row
- Reads every cached key and value
- Cache grows by one K, V per token

▼ Each decode step reads the whole cache

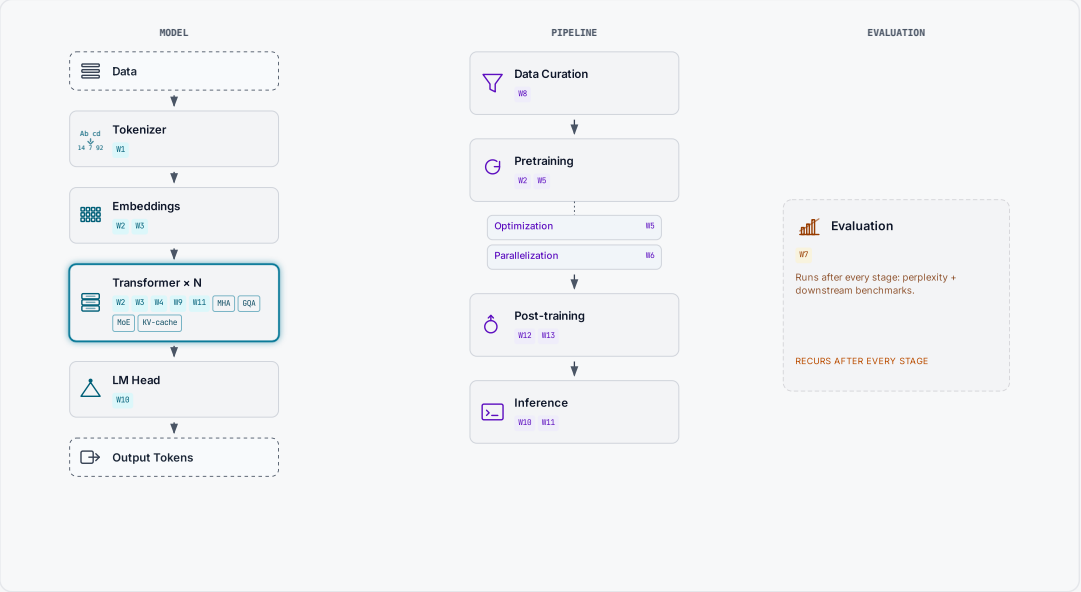
In Week 3's terms, prefill computes the whole prompt in one pass and fills the KV cache. Each decode step then handles a single new token. In every layer, its key and value join the cache, and its query forms the one new row of scores, against every cached key. Rows from earlier steps are neither kept nor rebuilt. Week 3 shows why the causal mask keeps the cached keys and values exact.

The widget opens on decode step 1 of Week 3's example sentence. The prompt is "The cat sat on", and the step feeds "the", the fifth token. The step computes one query, one key and one value, its new row holds 5 scores, and it takes 4 keys and 4 values from the cache. Filled chips mark what the step computes, and outlined chips what it reads from the cache. All counts are per head and per layer. The split into prefill and decode, and the marking of each key and value as computed or cached, follow a figure by Verma and Vaidya (Verma & Vaidya, 2023).

With full causal attention, each decode step therefore reads the whole cache, in every layer, and the cache it reads grows by one key and one value per token, per layer and per KV head. GQA and MLA change what a token stores. A sliding window or learned sparsity changes which tokens a query reads.

Where this week's code goes Read on your own

- Window: `causal_mask` and per-layer `layer_types`
- MLA beside Week 3's attention
- Sinks: evaluate a prepared attention trace
- Backends measured against the roofline



10 CONNECTIONS

◀ Prev Reset Start ▶

Click "Start" to walk through how decisions in one part of the pipeline shape every other part.

▼ Where the exercise changes the code

The map places this week in the course. Its highlighted stage is the Transformer × N in the model column, because all of this week's model changes are inside attention. Its connection from the Transformer to Inference reads "Attention variant (MHA / GQA / MLA) sets the KV-cache footprint": the attention variant decides how much each token stores, and inference pays for that storage. The map is for navigation and carries no measurements.

The exercise turns the lecture into code in four pieces:

1. the exact inclusive sliding-window boundary in `causal_mask`, with a per-layer `layer_types` setting that interleaves local and full layers;
2. the unabsorbed course form of MLA as a second attention class beside `CausalSelfAttention`, with a check of how many numbers each token stores;
3. perplexity on a prepared attention trace under the masks it defines, set beside the full-context perplexity;
4. attention profiled under named backends and placed against the roofline.

The Memory Wall

Every kernel computes and moves bytes

resource lower bound

$$T \geq \max\left(\frac{F}{P_{\text{peak}}},\ \frac{B}{\beta}\right)$$

- Compute time: FLOPs ÷ peak
- Movement time: bytes ÷ bandwidth
- The slower one binds

▼ A lower bound on time from two resources

A GPU kernel spends time on two things: the arithmetic it performs, and the data it moves between memory and the units that compute. Each has a rate limit.

Write F for the useful arithmetic in floating-point operations (FLOPs) and P_{peak} for the device's peak arithmetic rate in FLOP/s. Write B for the bytes that cross the boundary of one chosen memory tier and β for that tier's bandwidth in bytes per second. In this lecture's formulas B counts bytes, and the batch size is written out in words. In a tensor shape such as `[B,H,S,S]`, `B` still names the batch axis.

- **A1. Two rate limits.** The kernel performs F FLOPs at a rate of at most P_{peak}, and it moves B bytes across the chosen tier at a rate of at most β.
- **A2. Ideal overlap.** Arithmetic and movement run fully at the same time, so neither waits for the other.

Claim (lower bound). Under A1, the kernel's time T, in seconds, satisfies

$$T \geq \max\left(\frac{F}{P_{\text{peak}}},\ \frac{B}{\beta}\right).$$

The arithmetic alone takes at least F/P_{peak} and the movement alone at least B/β, so T is at least the larger of the two.

Both bounds hold whatever the overlap, so the expression is a lower bound, not a prediction of the latency. A kernel reaches it only under A2. In a real kernel arithmetic and movement overlap only in part, or they add up, and the time lies above the bound. Launch, dependency and synchronization costs are outside the expression altogether.

The slower resource binds: making the other one faster does not lower the floor. Which resource binds depends on the workload (its shape, its data type and how it is implemented) and on which memory tier you count. "Memory-bound" therefore describes a stated workload at a stated memory tier, and the name of an operation does not settle it. Use the comparison to decide what to inspect, then measure F, the traffic and the elapsed time over the same region of code, on the system you run.

The **memory wall** is the name for a trend: the compute available on a device grows faster than the rate at which memory can supply its data. Roofline analysis turns the comparison above into a calculation for one device.

8

Arithmetic intensity counts FLOPs per byte

arithmetic intensity

$$I = \frac{F_{\text{useful}}}{B_{\text{HBM}}}$$

- Bytes to and from high-bandwidth memory (HBM)
- Low **intensity**: bandwidth binds
- High intensity: compute can bind

▼ Intensity divides useful work by memory traffic

Definition. The arithmetic intensity of an operation is the useful arithmetic it performs for each byte it moves,

$$I = \frac{F_{\text{useful}}}{B_{\text{HBM}}},$$

in FLOP per byte. The numerator F_{useful} counts useful FLOPs under a named counting convention. The usual one counts a multiply and an add as two FLOPs. The denominator B_{HBM} counts the bytes that cross the boundary of high-bandwidth memory (HBM), the GPU's large off-chip memory, in the read and write directions you state.

The bytes depend on a traffic model: the same operation reaches different intensities at different shapes and data types, and under different assumptions about fusion and reuse. So fix the workload shape, the data type (BF16 throughout this lecture), and the fusion and reuse assumptions, and make the numerator and the denominator count the same region of code.

Intensity decides which term of the bound is larger

Take HBM as the memory tier of the lower bound, so that $F = F_{\text{useful}}$, $B = B_{\text{HBM}}$ and β is the HBM bandwidth.

Claim. For an operation with $F > 0$, the movement term B/β is larger than the compute term F/P_{peak} if and only if $I < P_{\text{peak}}/\beta$.

The ratio of the two terms shows it:

$$\begin{aligned} \frac{B/\beta}{F/P_{\text{peak}}} &= \frac{B}{F} \cdot \frac{P_{\text{peak}}}{\beta} && \text{rearrange} \\ &= \frac{P_{\text{peak}}/\beta}{I} && \text{definition of } I. \end{aligned}$$

The ratio is above 1, and movement takes longer, exactly when the intensity is below P_{peak}/β, a threshold fixed by the device alone. At low intensity bandwidth therefore binds. At high intensity compute can bind, though a real kernel may still fall short of the compute peak.

9

An H100 SXM, in BF16:

How many FLOPs per byte keep it busy?
About 3, 30, 300 or 3,000?

Answer A1

The H100 SXM is our reference GPU

H100 SXM reference ridge

$$I_{\text{ridge}} = \frac{989.5 \times 10^{12}}{3.35 \times 10^{12}} = 295.37 \text{ FLOP/byte}$$

- 132 streaming multiprocessors (SMs)
- Shared memory: 228 KB per SM
- L2 cache: 50 MB
- HBM3: 80 GB at 3.35 TB/s (NVIDIA, 2026a) (NVIDIA, 2026b) (NVIDIA, 2026c)
- BF16: 989.5 TFLOP/s dense, 1,979 only with 2:4 sparsity
- Profile your assigned GPU

▼ The reference numbers and the conventions they carry

The ridge in decimal units

The formula divides the dense-equivalent BF16 Tensor Core peak, $P_{\text{peak}} = 989.5 \times 10^{12}$ FLOP/s, by the HBM3 bandwidth, $\beta = 3.35 \times 10^{12}$ byte/s (NVIDIA, 2026a). Both rates are in decimal units, with tera standing for 10^{12}, so the ridge of the H100 SXM in dense BF16 is

$$I_{\text{ridge}} = \frac{989.5 \times 10^{12} \text{ FLOP/s}}{3.35 \times 10^{12} \text{ byte/s}} = 295.373\ldots \text{ FLOP/byte}.$$

Dense and sparse peaks

The peak follows a convention. NVIDIA's headline for BF16 on the H100 SXM is 1,979 TFLOP/s, quoted for 2:4 structured sparsity, a fine-grained sparsity feature that doubles the throughput of standard Tensor Core operations (NVIDIA, 2026b). Tensor Cores are the GPU units that execute matrix multiplications. Without sparsity the peak is half the headline, which gives the dense-equivalent peak (NVIDIA, 2026a):

$$P_{\text{peak}} = \frac{1{,}979}{2} \text{ TFLOP/s} = 989.5 \text{ TFLOP/s}.$$

A sparse peak applies only to an operation that satisfies the sparsity convention. Dense attention does not, so it is compared with the dense-equivalent peak.

The memory hierarchy

The GPU's memory is a hierarchy of tiers, each with its own capacity and bandwidth. The H100 SXM has 132 streaming multiprocessors (SMs), the units that run a kernel's threads. Each SM has its registers and up to 228 KB of shared memory, small on-chip storage for working data. A 50 MB L2 cache serves all the SMs. Off the chip, 80 GB of HBM3 supply data at 3.35 TB/s (NVIDIA, 2026a) (NVIDIA, 2026b) (NVIDIA, 2026c).

Registers and shared memory give the closest reuse. L2 can keep data for reuse across the blocks of threads that run on different SMs. Requests that none of these on-chip tiers serve go to HBM and count as HBM traffic.

Capacity says whether a working set fits in a tier, and bandwidth says how fast that tier serves transfers.

What a hardware number must name

A hardware number is therefore incomplete until it names five things: its form factor, its precision, whether it is theoretical or measured, its memory tier, and whether it counts dense or sparse operations. This reference keeps them together: the H100 SXM form factor, BF16 as precision and data type, the dense-equivalent peak, and HBM3 as the memory tier, with both read and write traffic across its boundary.

Profile the GPU you are assigned and report the rates it achieves. The course's UC3 queues allocate an H100 SXM5 with 94 GB of HBM, which `nvidia-smi` reports as "NVIDIA H100" with 95,830 MiB of memory. It is not the 80 GB part these numbers describe. It has the same SMs and SM clock, so its dense BF16 peak is the same, but its memory differs, and so does its bandwidth. No specification sheet lists this part, so the exercise has you compute its HBM bandwidth from the memory clock and the bus width, and record where each of your two peaks came from.

Operation intensity depends on the traffic model

GEMM, each tensor crosses HBM once

$$I_{\text{GEMM}} = \frac{2MKN}{b\,(MK + KN + MN)}$$

- Elementwise, BF16: 0.25 FLOP/byte
- Matrix-vector at batch 1: about 1, weights dominate
- Softmax, norm: about 2, depends on passes and fusion
- Square matmul, n = 4,096: 1,365
- Only the matmul clears the ridge of 295

▼ Deriving the four intensities

Each intensity follows from counting the useful FLOPs and the bytes of a stated traffic model. Two assumptions hold for all four operations.

- **A1. BF16 elements.** Every element takes $b = 2$ bytes, because BF16 is the data type throughout this lecture. Here b counts bytes per element, not a batch size or Week 3's RoPE base.
- **A2. Multiply-add convention.** A multiply and an add count as two FLOPs.

Elementwise: 0.25 FLOP/byte

An unfused unary elementwise operation, such as adding a constant to every element, does one useful FLOP per element.

- **A3. One read and one write.** Each element is read once from HBM and its result written once, with nothing fused.

Then

$$I_{\text{elem}} = \frac{1 \text{ FLOP}}{2 \text{ bytes read} + 2 \text{ bytes written}} = 0.25 \text{ FLOP/byte}.$$

Matrix-vector product: about 1 FLOP/byte

A matrix-vector product (GEMV) at batch size 1 multiplies each weight by one activation and adds the result to a sum.

- **A4. Weight traffic dominates.** Only the weights are counted. The traffic of the input and output vectors is left out.

Then

$$\begin{aligned} I_{\text{GEMV}} &\approx \frac{2 \text{ FLOP per weight}}{b \text{ bytes per weight}} && \text{A2, A4} \\ &= 1 \text{ FLOP/byte} && \text{A1: } b = 2. \end{aligned}$$

Softmax and normalization: about 2 FLOP/byte

A softmax or a normalization has no single intensity. It is

$$I_{\text{row}} = \frac{F_{\text{row}}}{B_{\text{row}}},$$

the FLOPs and the HBM bytes for one row. Both depend on how many passes the implementation makes over the row and on what it fuses.

- **A5. The widget's pass model.** The value 2 on the chart is the widget's teaching assumption for one such implementation, with its pass count and fusion fixed.

General matrix multiplication: $2n/(3b)$

A general matrix multiplication (GEMM) multiplies an $M \times K$ matrix by a $K \times N$ matrix, `[M,K]@[K,N]`. It computes MN outputs, each from K multiplications and K additions.

- **A6. One crossing per tensor.** Each input is read from HBM once and the output is written once. This one-crossing model assumes ideal reuse.

Under A1, A2 and A6,

$$
\begin{aligned}
F &= 2MKN && MN \text{ outputs, } 2K \text{ FLOPs each, A2} \\
B &= b\,(MK + KN + MN) && \text{A6} \\
I_{\text{GEMM}} &= \frac{2MKN}{b\,(MK + KN + MN)} && I = F/B,
\end{aligned}
$$

which is the formula on the slide. For a square product, with $M = K = N = n$,

$$
\begin{aligned}
I_{\text{GEMM}} &= \frac{2n^3}{3bn^2} = \frac{2n}{3b} && M = K = N = n \\
&= \frac{8{,}192}{6} && n = 4{,}096,\ b = 2 \\
&= 1{,}365.333\ldots \ \text{FLOP/byte.}
\end{aligned}
$$

The four operations against the ridge

Operation	Traffic assumption	Intensity (FLOP/byte)	Against the ridge of 295
Unary elementwise, unfused	one read and one write per element (A3)	0.25	below
GEMV at batch size 1	weights only (A4)	about 1	below
Softmax or normalization	the widget's pass model (A5)	about 2	below
Square GEMM, $n = 4{,}096$	each tensor crosses HBM once (A6)	1,365.33	above

On the H100 SXM, three of the four operations are memory-bound, and only the square matmul clears the ridge. Fusion, batch size, data type and shape change both the useful work and the traffic, so the ordering belongs to these four traffic models. Check the analytic values against measurements of the same synchronized code.

Compute has outpaced off-chip bandwidth Read on your own

- A100 → H100: compute ×3.2 (NVIDIA, 2026b)
- Off-chip bandwidth: only ×1.9
- Ridge: 201 → 333 FLOP/byte
- Recompute the ridge per device

▼ One matched comparison of two generations

Reported. NVIDIA's Hopper architecture description compares the A100 and the H100 in one table of preliminary specifications, its Table 3 (NVIDIA, 2026b). There the A100 SXM4 is listed at 312 TFLOP/s in dense BF16 with an off-chip bandwidth of 1.555 TB/s, and the H100 SXM5 at 1,000 TFLOP/s and 3.000 TB/s. The growth factors and the two ridges, in FLOP/byte, follow:

$$\begin{aligned} \text{compute growth} &= \frac{1{,}000}{312} = 3.205\ldots \\ \text{bandwidth growth} &= \frac{3.000}{1.555} = 1.929\ldots \\ I_{\text{ridge}}^{\text{A100}} &= \frac{312}{1.555} = 200.64\ldots \\ I_{\text{ridge}}^{\text{H100}} &= \frac{1{,}000}{3.000} = 333.33\ldots \end{aligned}$$

Compute grows by about $3.2\times$ and bandwidth only by about $1.9\times$, so the ridge moves from about 201 to about 333 FLOP/byte. These preliminary values are matched with each other. They differ from the later H100 product reference of 989.5 TFLOP/s and 3.35 TB/s, whose ridge is 295 FLOP/byte, so keep the two sets apart.

Over a longer span of hardware generations, compute has often grown faster than off-chip memory (DRAM) bandwidth (Gholami et al., 2024). That trend is the memory wall. A newer GPU need not move its ridge to the right, though. Calculate the ratio of compatible peak to bandwidth for the device and convention at hand, and reduce the traffic when an operation stays below it.

Model FLOPs utilization (MFU) compares model work with peak

single-device MFU

$$P_{\text{model}} = F_{\text{model/token}} \times R_{\text{token}}; \quad \text{MFU}_{1\text{GPU}} = \frac{P_{\text{model}}}{P_{\text{peak,matching}}}$$

- Model FLOPs utilization (MFU), from PaLM (Chowdhery et al., 2023)
- 80 GFLOP/token × 8,000 tokens/s
- = 640 TFLOP/s, 64.68% of the 989.5 peak
- Covers the whole model end to end

▼ One MFU calculation and the conventions behind it

Definition. Model FLOPs utilization (MFU), defined in the PaLM paper, divides the rate of useful model work by a matching hardware peak (Chowdhery et al., 2023). The model's rate is

$$P_{\text{model}} = F_{\text{model/token}} \times R_{\text{token}},$$

where $F_{\text{model/token}}$ is the model FLOPs per token under a stated counting convention and R_{token} is the number of tokens processed per second. On one GPU,

$$\text{MFU}_{1\text{GPU}} = \frac{P_{\text{model}}}{P_{\text{peak,matching}}}.$$

An illustrative calculation

Take an illustrative run. Its counting convention gives 80 GFLOP per model token and leaves out optimizer work, and a synchronized measurement gives 8,000 model tokens per second. The matching peak is the dense-equivalent BF16 Tensor Core peak of one H100 SXM, 989.5 TFLOP/s (NVIDIA, 2026a), because the run is dense and in BF16. The model then runs at 640 TFLOP/s, and its single-device MFU is 64.68%:

$$P_{\text{model}} = 80 \times 10^9 \times 8{,}000 = 640 \times 10^{12}\ \text{FLOP/s},$$
$$\text{MFU}_{1\text{GPU}} = \frac{640}{989.5} = 0.646791\ldots = 64.68\%.$$

MFU measures the whole model end to end. A kernel's efficiency needs its own measurement. Two MFU values compare only when their conventions match. A different token or FLOP convention, timing region, device count, precision, or dense or sparse peak gives a different number for the same run.

Achieved bandwidth from the same rate

The same token rate gives an estimate of the achieved bandwidth,

$$B_{\text{traffic/token}} \times R_{\text{token}},$$

where $B_{\text{traffic/token}}$ is the modeled traffic per token in bytes. The estimate holds only when the traffic model and the token rate cover the same boundary and the same synchronized region, and it comes from the model, not from hardware counters. Week 6 extends MFU to several GPUs.

Naive attention is memory-bound here

- S × S scores written to HBM once, read twice
- S = 4,096, 32 heads: 3 GiB moved
- 85 FLOP/byte: below the ridge

▼ Counting the traffic of the score matrix

A naive attention implementation computes the scores $QK^\top$ and writes the full $S \times S$ score matrix of every head to HBM. It reads the matrix back to compute the softmax, and reads it again for the product with the values. Count those bytes for the shape and the backend you run, use the compute and traffic model to predict which resource binds, then profile.

- **A1. Score traffic only.** The traffic model counts one write of the score tensor and two reads of it.
- **A2. The shape.** One sequence (batch size 1), $H = 32$ heads, $S = 4{,}096$ tokens and head width $D_h = 128$, in BF16 at 2 bytes per element. The score tensor has shape `[B,H,S,S]`, here `[1,32,4096,4096]`.
- **A3. Two matrix products.** The FLOPs counted are those of $QK^\top$ and of the product with V. Each has HS^2D_h multiply-adds for one sequence, so each takes $2HS^2D_h$ FLOPs.

Claim. Under A1 to A3, one score tensor takes 1 GiB, the score traffic is 3 GiB, the two products take about 275 GFLOP, and the intensity is 85.33 FLOP/byte.

The bytes

The score tensor holds

$$\begin{aligned} HS^2 &= 32 \times 4{,}096^2 && \text{A2} \\ &= 2^5 \times 2^{24} = 2^{29} = 536{,}870{,}912 \end{aligned}$$

elements, so the traffic is

$$\begin{aligned} \text{one tensor} &= 2 \times 2^{29} \text{ bytes} && \text{A2: 2 bytes each} \\ &= 2^{30} \text{ bytes} = 1 \text{ GiB} \\ B &= 3 \times 2^{30} \text{ bytes} && \text{A1: one write, two reads} \\ &= 3 \text{ GiB}. \end{aligned}$$

The FLOPs and the intensity

The FLOPs, in decimal GFLOP, are

$$\begin{aligned} F &= 2 \times 2HS^2D_h && \text{A3, two products} \\ &= 2^2 \times 2^5 \times 2^{24} \times 2^7 = 2^{38} && \text{A2} \\ &= 274.877906944 \text{ GFLOP}. \end{aligned}$$

Against the score traffic alone, the intensity is

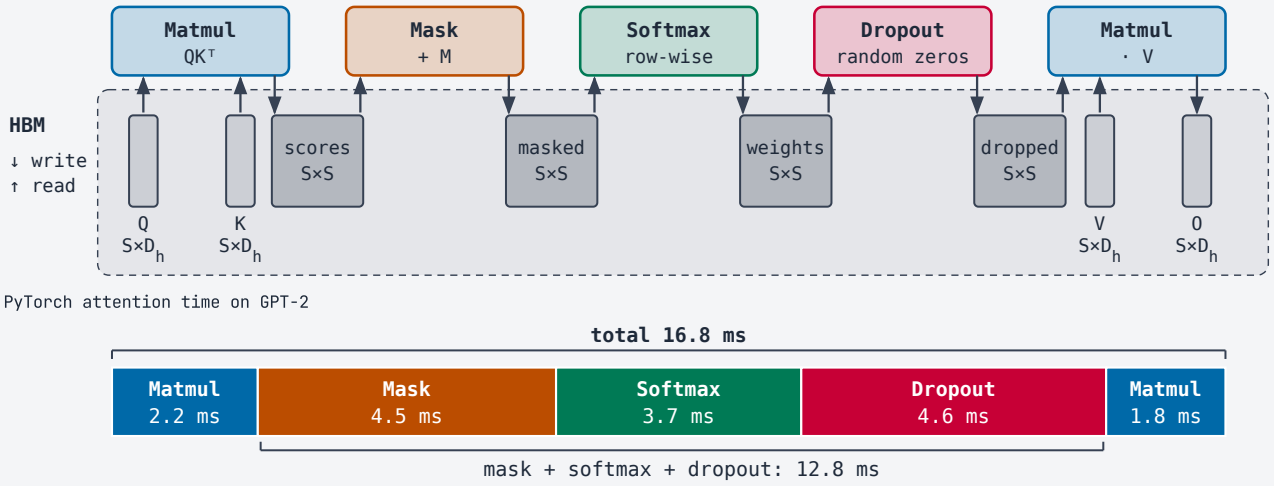
$$\begin{aligned} I &= \frac{2^{38}}{3 \times 2^{30}} && I = F/B \\ &= \frac{2^8}{3} = 85.333\ldots \text{ FLOP/byte}. \end{aligned}$$

That is well below the H100 SXM ridge of 295 FLOP/byte, so under this traffic model, at this shape, naive attention is memory-bound at HBM.

The traffic model leaves out the Q, K, V and output projections, the reads of Q, K and V, further softmax passes, masks, the backward pass, cache effects, allocator behavior and fusion in the backend. A real backend may fuse operations or avoid some of these transfers.

FlashAttention

Naive attention mostly moves S × S tensors



Dao et al., 2022

Top: a schematic, with tensor sizes not to scale. Q, K, V and the output are S by head width, and the four intermediates are S×S. Bottom: redrawn from Figure 1 (right) of Dao et al., which prints no values. Each segment was read from the vector figure and comes out as a whole tenth of a millisecond. The paper does not state the sequence length, batch size, GPU, mask, dropout probability or pass (forward, or forward and backward) behind this bar. The figure also shows a FlashAttention bar, which is not drawn here.

- One kernel per step, each writing to HBM and reading back
- Mask, softmax, dropout: 12.8 of 16.8 ms (Dao et al., 2022)

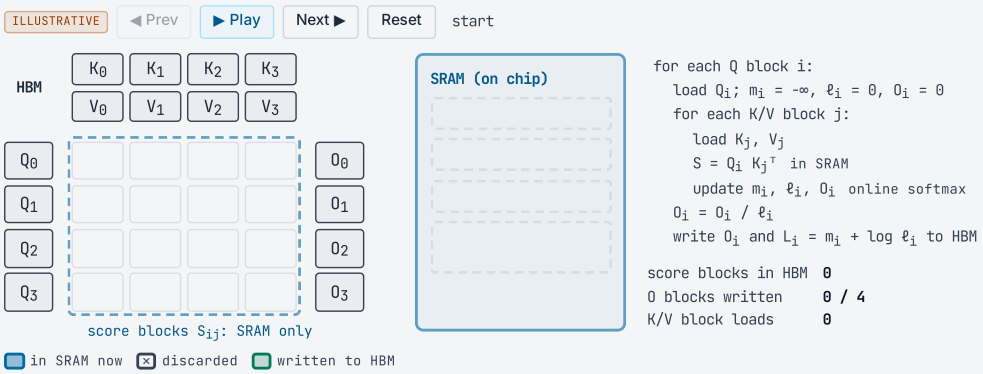
▼ Five kernels, four S × S round trips

Week 2 computed attention in four steps: the scores $QK^\top$, the mask, the softmax and the weighted sum of the values. A naive implementation launches one GPU kernel per step, and each kernel writes its result to HBM for the next one to read back. GPT-2, the model in the figure, adds a fifth kernel for dropout on the attention weights during training. Four of the five results are $S \times S$: the scores, the masked scores, the weights and the weights after dropout. The slide "Naive attention is memory-bound here" counted the bytes of this traffic without the mask and dropout kernels.

The bar redraws Figure 1 (right) of Dao et al., PyTorch's attention time on GPT-2 by kernel (Dao et al., 2022). The two matrix products do almost all of the arithmetic and take 4.0 ms. Mask, softmax and dropout do a few operations per element, but each reads an $S \times S$ tensor from HBM and writes one back. Together they take 12.8 of the 16.8 ms, the time it takes to move their tensors. The course's model has no dropout, and the benchmark behind the opening puzzle ran without it.

The same figure reports FlashAttention's fused kernel as up to 7.6 times faster, the paper's result for attention on GPT-2 (Dao et al., 2022). That is a different measurement from the threefold speedup of the opening puzzle. It leaves one question: can exact attention be computed without its $S \times S$ tensors making these trips to HBM?

Three ideas make FlashAttention work



Watch three things. The score blocks S_{ij} never reach HBM: each one is computed in SRAM, folded into the running state and thrown away. Each O block is written to HBM once, after its Q block has met every K/V block. K and V are read again for every Q block, 16 block loads in all. The loop order is FlashAttention-2's, with the Q blocks outside. FlashAttention-1's Algorithm 1 swaps the two loops. The four-by-four block count is illustrative, and so are the steps where the running maximum rises.

- Tiling: K/V blocks stream through on-chip static RAM (SRAM) (Dao et al., 2022)
- Online softmax: running max `m`, sum `ℓ`, output `o`
- Backward: recompute blocks from a row statistic

▼ Tiling, online softmax and recomputation

FlashAttention-1 (FA1) combines three ideas: tiling, online softmax and recomputation (Dao et al., 2022). FlashAttention-2 (FA2) keeps them and swaps the loops (Dao, 2024). The widget and this note follow FA2's forward pass, in the papers' symbols: N for the sequence length (the course's S) and d for the head width (the course's D_h).

Tiling

The first idea is **tiling**. The kernel splits Q into blocks of B_r rows, and K and V into blocks of B_c rows, small enough for on-chip static RAM (SRAM): the registers and shared memory of each SM from the Memory Wall tab, up to 228 KB per SM on the H100 SXM. For each Q block, FA2's inner loop streams every K/V block through SRAM, computes that block of scores, folds it into a running state and discards it. The finished output block goes to HBM once, and the score matrix never does. FA1's Algorithm 1 puts the K/V blocks in the outer loop, so it reads and writes each output block and its running state once per K/V block (Dao et al., 2022).

Why streaming needs an online softmax

A softmax needs the maximum and the sum over a whole row, but the kernel sees one block at a time. Fix one query row, with scores $s_1, \dots, s_N$ and value vectors $v_1, \dots, v_N \in \mathbb{R}^d$. Its output is the softmax-weighted sum of the values,

$$y = \sum_{j=1}^{N} p_j \, v_j, \tag{1}$$

where the weight of key j is its softmax probability

$$p_j = \frac{e^{s_j}}{\sum_{j'=1}^{N} e^{s_{j'}}}.$$

Subtracting the same constant m from every score leaves this ratio unchanged, because e^{-m} cancels. Implementations subtract the row maximum so that no exponential overflows, but that maximum is known only after the last block.

The running state and its update

For each query row, the kernel therefore keeps three running quantities:

- the running maximum m, the largest score seen so far;
- the normalizer ℓ, the running sum $\sum e^{s_j - m}$ over the keys seen so far;
- the output accumulator o, the running sum $\sum e^{s_j - m} v_j$, not yet divided by ℓ.

For a Q block of B_r rows, m and ℓ have shape `[B_r]` and o has shape `[B_r, d]`. Before the first block the state is neutral: $m = -\infty$ and $\ell = o = 0$.

When a block J of keys arrives, the kernel replaces the old state by

$$\begin{aligned} m^{\text{new}} &= \max\bigl(m^{\text{old}},\ \max_{j\in J} s_j\bigr), \\ \ell^{\text{new}} &= e^{m^{\text{old}}-m^{\text{new}}}\,\ell^{\text{old}} + \sum_{j\in J} e^{s_j-m^{\text{new}}}, \\ o^{\text{new}} &= e^{m^{\text{old}}-m^{\text{new}}}\,o^{\text{old}} + \sum_{j\in J} e^{s_j-m^{\text{new}}}\,v_j. \end{aligned} \tag{2}$$

The factor $e^{m^{\text{old}}-m^{\text{new}}}$ turns every old term $e^{s_j-m^{\text{old}}}$ into $e^{s_j-m^{\text{new}}}$, so the update rescales the old state and then adds the new block's terms. The factor is 1 when the block does not raise the maximum and below 1 when it does. On the first block it is $e^{-\infty}=0$ and multiplies the zero state.

Why the result is exact

After every block, the state is the softmax state of the keys seen so far, measured against their largest score, because one factor moves all old terms to the new maximum. After the last block the sums run over all N keys, m is the row maximum, and the common factor e^{-m} cancels:

$$\frac{o}{\ell} = \frac{\sum_{j=1}^{N} e^{s_j-m}\,v_j}{\sum_{j=1}^{N} e^{s_j-m}} = \frac{\sum_{j=1}^{N} e^{s_j}\,v_j}{\sum_{j=1}^{N} e^{s_j}} = y.$$

This holds for any split of the row into blocks, so in exact arithmetic the block size does not change the output. In floating point, the partial sums are added in a different order than in a one-pass softmax, so the outputs can differ in their last digits. "Exact" therefore means the same mathematical function, not bitwise-equal outputs.

FA2 divides o by ℓ once, after the last block, to save operations other than matrix multiplications (Dao, 2024). FA1's Algorithm 1 normalizes the output after every block. In exact arithmetic the results are the same.

Recomputation in the backward pass

The third idea is **recomputation**. The backward pass needs the attention probabilities, which standard attention reads back from HBM. FlashAttention stores a statistic per query row instead and recomputes the score and probability blocks in SRAM from Q, K and that statistic. FA1 stores m and ℓ (Dao et al., 2022). FA2 stores one number, the logsumexp $L = m + \log \ell$ (not the number of layers) (Dao, 2024). With the row's final m and ℓ,

$$p_j = \frac{e^{s_j-m}}{\ell} = e^{s_j-L},$$

so the stored statistic turns a recomputed score s_j into its probability.

What the widget adds

Beside the block grid, the SRAM panel holds the running state of the current Q block as m_i, ℓ_i and O_i. At some compute steps it reports that m rose, the rescale of (2). At the end of each Q block it divides O_i by ℓ_i and forms L_i before both go to HBM.

One row, worked through

Take one query row with eight keys in four blocks of two, and scalar values, so o is a number. The scores and values are illustrative. The first block, with scores $(1.0,\ 2.0)$ and values $(0.4,\ 0.8)$, gives

$$\begin{aligned} m &= 2.00, \\ \ell &= e^{1-2} + e^{2-2} \approx 1.37, \\ o &= 0.4\,e^{-1} + 0.8\,e^{0} \approx 0.95. \end{aligned}$$

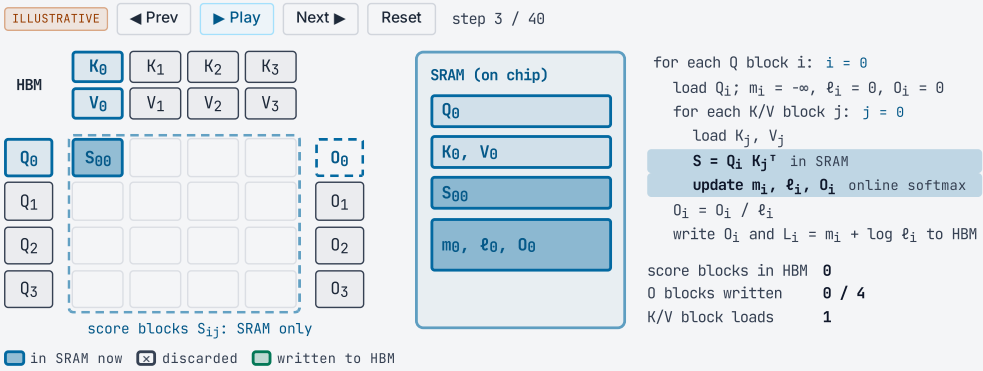
The second block raises the maximum from 2 to 3, so the old ℓ and o are multiplied by $e^{2-3} \approx 0.37$ before its terms are added:

$$\begin{aligned} \ell &= e^{-1}\cdot 1.37 + e^{0} + e^{-1.5} \approx 1.73, \\ o &= e^{-1}\cdot 0.95 + 1.2\,e^{0} + 0.6\,e^{-1.5} \approx 1.68. \end{aligned}$$

The table lists the state after each block. The factor drops below 1 where a block raises the maximum and stays at 1 where it does not.

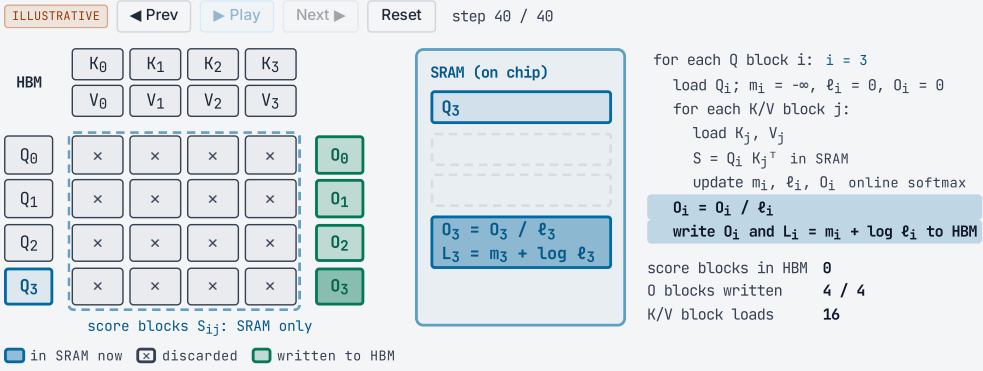
Block	Scores	Values	m	Factor	ℓ	o
Start			$-\infty$		0	0
1	$1.0,\ 2.0$	$0.4,\ 0.8$	2.00	$e^{-\infty} = 0$	1.37	0.95
2	$3.0,\ 1.5$	$1.2,\ 0.6$	3.00	$e^{-1} \approx 0.37$	1.73	1.68
3	$2.5,\ 4.0$	$1.0,\ 1.6$	4.00	$e^{-1} \approx 0.37$	1.86	2.44
4	$3.5,\ 2.0$	$1.4,\ 0.9$	4.00	$e^{0} = 1$	2.60	3.41

After the last block the row's output is $o/\ell = 3.41/2.60 \approx 1.31$. A softmax over all eight scores at once, taken against the row maximum 4, gives the same two sums, 3.41 and 2.60.



Watch three things. The score blocks S_{ij} never reach HBM: each one is computed in SRAM, folded into the running state and thrown away. Each O block is written to HBM once, after its Q block has met every K/V block. K and V are read again for every Q block, 16 block loads in all. The loop order is FlashAttention-2's, with the Q blocks outside. FlashAttention-1's Algorithm 1 swaps the two loops. The four-by-four block count is illustrative, and so are the steps where the running maximum rises.

Step 3: Q block 0 and K/V block 0 are loaded, and the first score block S00 exists only in SRAM while O0 starts to accumulate. The counter of score blocks in HBM reads 0 and stays there.



Watch three things. The score blocks S_{ij} never reach HBM: each one is computed in SRAM, folded into the running state and thrown away. Each O block is written to HBM once, after its Q block has met every K/V block. K and V are read again for every Q block, 16 block loads in all. The loop order is FlashAttention-2's, with the Q blocks outside. FlashAttention-1's Algorithm 1 swaps the two loops. The four-by-four block count is illustrative, and so are the steps where the running maximum rises.

Step 40: all four O blocks are written, each once, and no score block ever reached HBM. K and V were loaded 16 times, once for every Q block, which is the price of keeping the Q block in SRAM.

Exact attention without the score matrix

FA1 I/O result

$$\begin{aligned}\text{FlashAttention-1:}&\quad \Theta(N^2d^2/M)\\ \text{standard attention:}&\quad \Theta(Nd+N^2)\end{aligned}$$

- `N` sequence length, `d` head width, `M` SRAM size (Dao et al., 2022)
- FlashAttention-2's loop order keeps this count
- Counts HBM accesses, not runtime

▼ How many HBM accesses FlashAttention needs

Dao et al. count memory traffic in a model of the GPU with two levels, HBM and SRAM, as the number of elements moved between them in the forward pass (Dao et al., 2022). Their M is the size of SRAM in elements. It has nothing to do with the GEMM dimension M of the Memory Wall tab. For $d \le M \le Nd$ they prove these counts of HBM accesses:

$$\begin{aligned}\text{standard attention:}&\quad \Theta(Nd+N^2),\\ \text{FA1:}&\quad \Theta(N^2d^2/M).\end{aligned}$$

The notation $\Theta(g)$ states how a count grows, up to constant factors: for large inputs the count lies between two fixed positive multiples of g. In the standard count, the N^2 term counts the $N \times N$ score and probability matrices, which standard attention writes to HBM and reads back, and the Nd term covers reading Q, K and V and writing the output. FA1's count also grows with N^2, but it carries the factor d^2/M, so it shrinks as the SRAM grows.

Inside this model and range, and keeping only the leading terms, the ratio of the two counts is

$$r \approx \frac{N^2d^2/M}{Nd+N^2} = \frac{d^2}{M}\cdot\frac{N}{N+d} \le \frac{d^2}{M}.$$

The factor $N/(N+d)$ approaches 1 when the sequence is much longer than the head width, so FA1 needs fewer accesses whenever d^2 is well below M, by about the factor M/d^2. For head dimensions of 64 to 128 and an SRAM of about 100 KB, the paper's figure for its A100-era GPUs, Dao et al. call d^2 many times smaller than M (Dao et al., 2022). At 2 bytes per element, 100 KB holds about 50,000 elements. That is about 12 times $d^2 = 4{,}096$ at $d = 64$, but only about 3 times $d^2 = 16{,}384$ at $d = 128$.

The count in FA2's loop order

The FA2 paper takes the block sizes as inputs and does not restate the count (Dao, 2024). Assume each Q block holds $B_r = \Theta(M/d)$ rows, so that it and its output accumulator fill a fixed share of SRAM, and that the K/V blocks are small enough for the rest. Each Q block is read once and each O block written once, $\Theta(Nd)$ accesses in all. K and V are read once per Q block, N/B_r times:

$$\frac{N}{B_r}\cdot 2Nd = \Theta\left(\frac{N^2d}{B_r}\right) = \Theta\left(\frac{N^2d^2}{M}\right),$$

where the last step uses $B_r = \Theta(M/d)$. That is the same order as FA1's count.

The paper's Proposition 3 adds that no exact attention algorithm achieves asymptotically fewer accesses, $o(N^2d^2/M)$, for every M in the range $d \le M \le Nd$ (Dao et al., 2022). All of these results count HBM accesses in the forward pass of this memory model. Projections, the backward pass, kernel launches and the choice of backend lie outside them, so they guarantee no running time.

Back to the puzzle.

**13% more FLOPs, yet three times faster:
where did the time go?**

Answer A2

19

Each generation follows the hardware bottleneck Read on your own

- FA2: better work split, A100 era (Dao, 2024)
- FA3: overlap movement and math, Hopper (Shah et al., 2024)
- FA4: rebalance for Blackwell's units (Zadouri et al., 2026)
- Same math, new bottleneck

▼ Later versions keep the function and change the kernel

FA1 and its successors compute the same mathematical attention function with the same three ideas. What changes from one version to the next is the kernel, which follows the hardware resource that binds on a newer GPU generation.

FA2, on A100-era hardware, partitions the work better and spends less time on operations other than matrix multiplications (Dao, 2024). FlashAttention-3 (FA3) targets Hopper GPUs such as the H100. It uses Hopper's asynchronous execution to overlap data movement with computation (Shah et al., 2024). FlashAttention-4 (FA4) targets Blackwell. It rebalances the work among three units: the Tensor Cores, the special-function units (SFUs), which evaluate functions such as the exponential, and shared memory (Zadouri et al., 2026).

20

SDPA dispatch and FlexAttention programmability differ Read on your own

- Scaled dot-product attention (SDPA): dispatches backends (PyTorch Contributors, 2026)
- FlexAttention: you program the mask (PyTorch Contributors, 2026)
- FA4 in FlexAttention does not mean SDPA runs FA4
- Record the requested and the selected backend

▼ Two PyTorch interfaces, and what a measurement records

PyTorch offers two interfaces for fast attention. The first, `torch.nn.functional.scaled_dot_product_attention` or scaled dot-product attention (SDPA), expresses standard attention and dispatches each call to one of its backends, separate implementations of the same function (PyTorch Contributors, 2026). The second, FlexAttention, makes attention programmable. You write functions that modify the scores or the mask, and FlexAttention runs them with optimized kernels where those kernels are supported (PyTorch Contributors, 2026).

The two interfaces choose their kernels separately. PyTorch documents a FlashAttention-4 path for FlexAttention (PyTorch Contributors, 2026). That path does not mean an SDPA call runs FA4, because SDPA makes its own dispatch decision. When you profile attention, record the backend you requested, and the backend that was selected or the failure if the request was refused. Profile the named backend at the named shape, and do not infer its runtime from a name in the FlashAttention lineage.

The exercise uses both interfaces for the sliding window. It states the window once in `causal_mask`, as a boolean mask in SDPA's polarity. It states it again as a FlexAttention mask function: `window_mask_mod(window)` returns `mask_mod(b, h, q_idx, kv_idx)`, which says whether query `q_idx` may read key `kv_idx`. On a batch without padding, the two must agree.

Sliding-Window Attention

21

Your KV cache is 8× too big.

You serve a model at 128k tokens of context.
What would you throw away or shrink?
What might break?

22

Three ways to keep or read fewer bytes

- Drop old tokens: **sliding window**
- Store less per token: MLA
- Read only selected tokens: **learned sparsity**

▼ Each mechanism changes a different quantity

The ideas for keeping or reading fewer of the KV cache's bytes fall into three groups, and each group is a mechanism in the rest of this lecture.

- **Drop old tokens.** A sliding window lets each query read only the most recent tokens. That is also what can break: a fact stated far back in the context is out of reach, so retrieval can suffer.
- **Store less for each token.** Storing fewer KV heads is Week 3's GQA. Multi-head latent attention (MLA) stores a smaller representation of each token instead.
- **Read only selected tokens.** Learned sparse attention keeps every token, so the cache does not shrink. The model chooses which entries each query reads.

Week 11 covers three further ideas for the cache: quantizing it, sharing it across layers, and evicting entries from it.

FlashAttention belongs to none of the three groups. It keeps every token and stores the same keys and values, so its cache still grows with the context. What it changes is the traffic: it reorganizes exact attention so that computing it moves fewer bytes. That matters because a raw FLOP count alone does not predict attention time. The resource that binds can be compute or data movement.

Each mechanism therefore changes a different quantity: a sliding window the reach, GQA and MLA the stored state, and learned sparsity the stored entries a query reads.

The Banded Mask, at Last

inclusive SWA boundary

$$\max(0, i - w + 1) \le j \le i$$

- `w = 1`: each token attends only to itself
- `w ≥ S`: full causal attention
- Mistral 7B: `w = 4,096` (Jiang et al., 2023)

▼ The window is one more condition in the mask

Dense causal attention lets each query use its full prefix. Sliding-window attention (SWA) shortens that reach.

The inclusive band

Number the positions of a sequence of length S from 0 to $S - 1$. Let i be the position of a query, j the position of a key, and $w \ge 1$ the width of the window.

Definition. With an inclusive window of width w, the query at position i may read the key at position j exactly when

$$\max(0,\ i - w + 1) \le j \le i. \tag{1}$$

The upper bound $j \le i$ is the causal boundary, so no query reads the future. The lower bound admits the w most recent positions, the query's own included, which is why this window is called inclusive. Near the start of the sequence, $i - w + 1$ is negative, and the $\max(0, \cdot)$ clips it to 0, so an early query reads only the positions that exist. The query at i therefore reads

$$\min(i + 1,\ w)$$

keys: all $i + 1$ positions of its prefix while $i < w$, and w keys from then on.

Two edge cases check the definition. With $w = 1$ the lower bound is $\max(0, i) = i$, so the bounds read $i \le j \le i$, and the query reads only its own key, $j = i$. With $w \ge S$, every query has $i - w + 1 \le 0$ because $i \le S - 1$, so the lower bound is 0 and the window recovers full causal attention.

Mistral 7B is a released model with a rolling window of $w = 4{,}096$ tokens (Jiang et al., 2023). Its paper writes the window as W and counts it the same way: with $W = 4{,}096$, each token attends to at most 4,096 positions, its own included, and its rolling cache holds 4,096 entries (Jiang et al., 2023).

The band in the mask

In code, the window is one more condition beside the Week 3 mask rules. The band is a predicate on the `[S, S]` grid of query and key positions. It is composed with those conditions and broadcast to the logical `[B, H, S, S]` mask of allowed pairs. With Week 3's sequence index b (not the bytes per element of the Memory Wall tab) and a head index h,

$$\begin{aligned}
&\text{allowed}[b, h, i, j] \\
&\quad = \text{valid_query}[b, i] \\
&\quad \wedge \text{valid_key}[b, j] \\
&\quad \wedge \text{same_document}[b, i, j] \\
&\quad \wedge \big(\max(0,\ i - w + 1) \le j \le i\big).
\end{aligned}$$

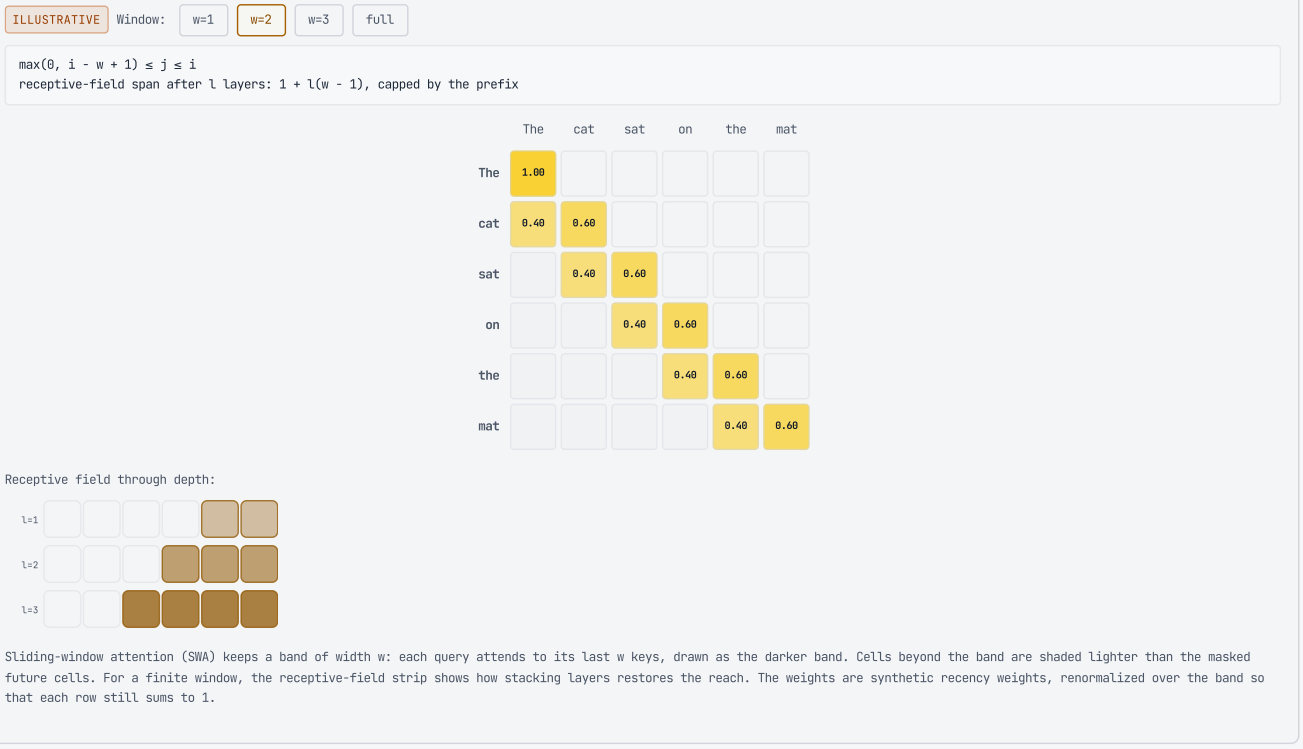
The last term is the band (1), and it supplies the causal boundary and the window together. The two validity terms remove padding, and the term same_document stops a query in one packed document from reading the keys of another. With $w \ge S$ the band admits the whole valid causal prefix, and the padding and document conditions still apply.

Depth Restores Reach

stacked receptive field

$$R_l = 1 + l(w-1)$$

- Reach is capped by the available prefix



▼ Each local layer adds at most w - 1 earlier positions

One windowed layer lets a position read only w positions. Stacking local layers grows the reach roughly linearly with depth, because each layer can pass on what the layer below gathered.

Definition. The stacked receptive field R_l is the number of consecutive input positions that a position can depend on after l local layers.

Assumptions

- **A1. The same inclusive window in every layer.** Each of the l layers uses the band (1) with the same width w. A full-attention layer would break this assumption.
- **A2. Attention is the only path between positions.** The rest of the block (residual connection, normalization and MLP) acts on each position separately, as in Weeks 2 and 3, so information moves between positions only through attention.

The reach grows by w – 1 per layer

Claim. Under A1 and A2, a position far enough from the start that no window is clipped has, after l local layers, the reach

$$R_l = 1 + l\,(w-1).$$

Before any layer, a position depends only on itself, so $R_0 = 1$. Layer l lets position i read the positions from $i - w + 1$ to i, and by A2 these are the only positions it gathers from. Each brings the range of input positions it gathered in the layers below. The range of position $i - w + 1$ ends $w - 1$ positions before the range of i, and neighboring ranges end one position apart, so together they form one consecutive stretch. Each layer therefore adds $w - 1$ positions, $R_l = R_{l-1} + (w-1)$, and unrolling from $R_0 = 1$ gives the claim.

The 1 is the current token, counted once. Each layer adds one position fewer than the window, because the window's own position is already in the count.

The cap by the prefix

A position cannot depend on more than its prefix. Near the start of the sequence the band's $\max(0, \cdot)$ clips the stretch at position 0, so position i depends on

$$\min\left(1 + l\,(w-1),\ i+1\right)$$

input positions. The reach grows linearly with depth until it fills the available prefix.

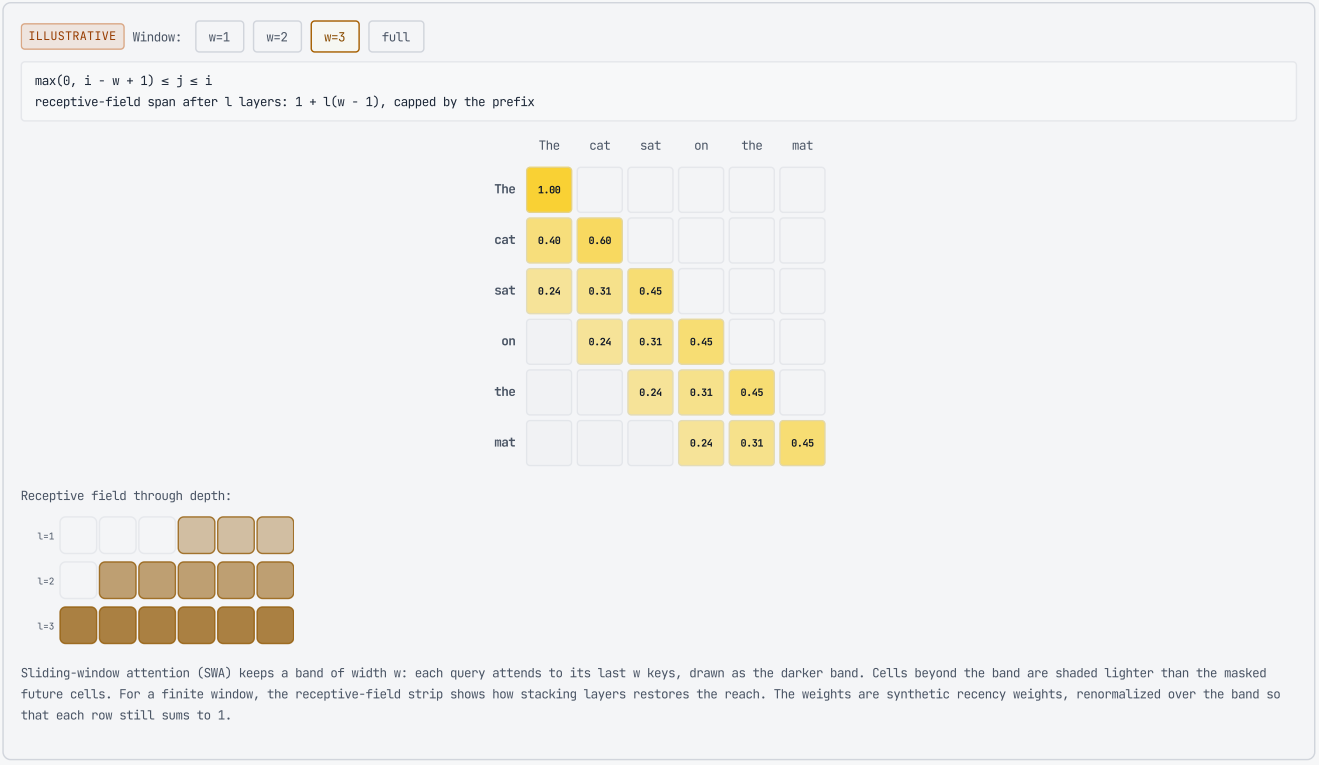
Reading the explorer

The `AttentionExplorer` widget shows a sequence of six tokens and opens at $w = 2$. Its grid is the mask of one layer: row i is a query, column j a key, and the band marks the keys the window admits. For a finite window, the strip under the grid shows the reach of the last token after one, two and three stacked layers. Each row reaches $w - 1$ positions further back than the row above, until the cap stops it at the first token. The cell values are a toy recency pattern on six tokens, and the explorer shows only which positions are reachable.

At $w = 3$ each layer adds two positions. The last token is at position 5, so its prefix holds six positions:

Layers l	$1 + l(w-1)$	Positions in the strip
1	3	3
2	5	5
3	7	6

The formula gives $R_3 = 7$, but the strip's third row stops at 6, the whole six-token prefix, because $\min(7, 6) = 6$.



At w = 3 each layer adds two positions: the last token reaches 3 positions after one layer and 5 after two. After three layers it would reach 7, but the strip stops at 6, the whole prefix.

Interleaving is a shipped pattern Read on your own

- MAI-Thinking-1: five local, one global (Microsoft AI, 2026)
- Local: GQA, window, RoPE
- Global: dense, NoPE
- One global layer sees everything again

▼ Local and global layers in one model

Some released models interleave local and global layers. MAI-Thinking-1 repeats a pattern of five local layers followed by one global layer, a ratio of 5:1. Its local layers use GQA, a sliding window and RoPE. Its global layers are dense, attending to the whole causal prefix, and use NoPE (Microsoft AI, 2026). Each kind of layer pairs one choice of position handling with one choice of locality. A global layer reads every earlier position, so after it every position can depend on its whole prefix again.

The exercise makes the locality choice per layer. Its `layer_types` setting lists for each layer `sliding_attention`, which uses the window, or `full_attention`, which reads the whole prefix. Position handling stays one setting for the whole model, as in Week 3. With `rope`, every layer rotates Q and K, the full layers included.

MAI-Thinking-1 is one dated configuration, from its 2026 report. Take it as an example, and choose the ratio and the pairing for another model on their own merits.

Mask Semantics, Kernel Work, and Cache Residency Are Separate

- Mask: decides which keys count
- `O(Sw)` work needs a kernel that skips masked blocks
- `O(w)` cache needs eviction that matches the window

▼ Three conditions behind a window's savings

Three claims about a window are easy to confuse, and each needs its own condition.

The mask, the work and the cache

The first claim is the meaning of the mask: which keys count. A mask that removes keys is called **semantic sparsity**. A dense masked implementation composes the `[S, S]` band into the `[B, H, S, S]` mask of allowed pairs and applies it to the full score matrix. The out-of-window scores are discarded, but a dense kernel may still compute every one of them, so the attention work stays $O(S^2)$.

The second claim is about the work. It falls to $O(Sw)$ only when the backend is window-aware and skips the out-of-window work instead of computing it. Skipping that work is **execution sparsity**.

The third claim is about the cache. During decoding, the cache of each layer falls to $O(w)$ only when an eviction policy limits the stored keys and values to the same visible set and no separate history is kept.

A skipping backend and matching eviction are separate conditions, one for the work and one for the cache. The window's semantics alone deliver neither saving.

How many pairs the band keeps

An analytic count shows how large the band is at one shape, $S = 32{,}768$ and $w = 4{,}096$, for one document without padding, so that the band (1) is the only condition that removes pairs. Query i then reads $\min(i+1, w)$ keys, and summing over the queries gives the visible query-key pairs:

$$\begin{aligned} C_{\text{band}} &= \sum_{i=0}^{S-1} \min(i+1,\ w) \\ &= \sum_{i=0}^{w-1} (i+1) + \sum_{i=w}^{S-1} w \\ &= \frac{w(w+1)}{2} + (S-w)\,w. \end{aligned}$$

At the shape,

$$\begin{aligned} C_{\text{band}} &= \frac{4096 \cdot 4097}{2} + 28{,}672 \cdot 4096 \\ &= 8{,}390{,}656 + 117{,}440{,}512 \\ &= 125{,}831{,}168, \end{aligned}$$

and dense causal attention has

$$\begin{aligned} C_{\text{causal}} &= \frac{S(S+1)}{2} = \frac{32{,}768 \cdot 32{,}769}{2} \\ &= 536{,}887{,}296 \end{aligned}$$

pairs. Their ratio is

$$\frac{C_{\text{band}}}{C_{\text{causal}}} = \frac{125{,}831{,}168}{536{,}887{,}296} \approx 0.23437,$$

so the band keeps 23.437% of the pairs, about 23%. The count covers pairs in the mask only, without the projections or hardware padding.

How many bytes the window cache holds

The cache has a matching count. For one sequence, the key-value payload of the decode cache is

$$2\,L\,H_{kv}\,D_h\,T_{\text{visible}}\,b \text{ bytes},$$

the formula of "Where Week 3 left off" with T_{visible} stored positions in place of S, times b bytes per element. Take an illustrative configuration with $L = 8$ layers, $H_{kv} = 2$ KV heads of width $D_h = 128$, and BF16 storage, so that $b = 2$. A cache that keeps only the window, $T_{\text{visible}} = w = 4{,}096$, holds

$$\begin{aligned} &2 \cdot 8 \cdot 2 \cdot 128 \cdot 4096 \cdot 2 \\ &\quad = 33{,}554{,}432 \text{ bytes}, \end{aligned}$$

which is 32 MiB. A cache that keeps all $T_{\text{visible}} = 32{,}768$ positions holds eight times as much, 256 MiB. Both figures are tensor sizes. A measured allocation adds allocator fragmentation and metadata, and neither figure includes other state or memory traffic.

27

A Window Only Pays If the Kernel Skips

Read on your own

- A dense kernel with a mask still does full work
- Fixed window: kept by rule
- Learned sparsity: the model chooses
- Local reach can cost retrieval quality

▼ Savings need a skipping kernel, and reach has a price

Whether a window saves work depends on the backend. A dense masked kernel still does dense work, so profile the named shape and backend to see whether the kernel skips the out-of-window tiles.

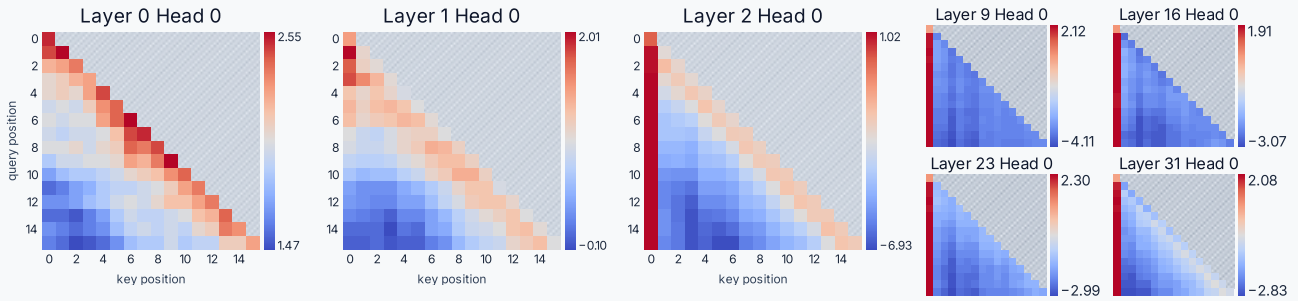
A fixed window and learned sparsity both read fewer keys. A fixed window keeps the w most recent positions by rule, whatever they contain. Learned sparsity lets the model choose, for each query, which tokens or blocks to read.

A window also trades quality for cost. A task that needs a fact from far back in the context can lose retrieval quality, by an amount that depends on the workload you run.

Attention Sinks

28

Some heads attend heavily to the first token



After Xiao et al., 2024, Figure 2 (values approximate)

A row is a query position and a column a key position, both counted from 0. Each panel has its own color scale, from blue at its lowest value to red at its highest, so compare colors only within a panel. The values are read off the colors of the published figure. The images under the labels Layer 16 Head 0 and Layer 23 Head 0 are titled layer 13 head 26 and layer 20 head 14 in the paper's source files.

- Llama-2-7B: early layers local (Xiao et al., 2024)
- Later layers: high logits on the first token

▼ **Above the first two layers, attention collects on the first token**

A sliding window keeps only the most recent positions, so it matters where a trained model's attention goes. Xiao et al. looked at the attention maps of Llama-2-7B. In layers 0 and 1 the patterns are mostly local, and queries attend mainly to recent keys. In the layers above them, they find strong attention logits on the initial token, across layers and heads. They call such positions **attention sinks**: positions on which some trained attention heads put a large share of their attention weight. In the models they study, the sinks are the initial tokens (Xiao et al., 2024).

The figure redraws the seven panels of their Figure 2, large for layers 0, 1 and 2 and smaller for layers 9, 16, 23 and 31. Each panel shows the average attention logits over 256 sentences of 16 tokens (Xiao et al., 2024). In layers 0 and 1 the high logits lie near the diagonal, on recent keys. From layer 2 on, the red first column holds the highest logit of every row after the first. A hatched cell is a future key that the causal mask removes, so it holds no logit.

Sun et al. report massive activations: a very few activations, at fixed positions, whose values are far larger than the others. In the layers after they appear, attention concentrates on the tokens that carry them, and in Llama-2 one of those tokens is the first (Sun et al., 2024).

Softmax must put its weight somewhere

normalization constraint

$$\sum_j \mathrm{softmax}(s)_j = 1$$

- Weights must sum to one
- An always-visible, low-impact position becomes a **sink**
- Proposed explanation, still a hypothesis (Xiao et al., 2024)

▼ Normalization suggests why sinks form, and the evidence stops short of proof

Xiao et al. propose an explanation for the first-token column that rests on one property of softmax.

Definition. For one query with score s_j on each key j it may read, softmax gives the weights

$$\mathrm{softmax}(s)_j = \frac{e^{s_j}}{\sum_k e^{s_k}},$$

where k runs over the same keys as j. Each weight is positive.

Claim. The weights of one query sum to one, whatever its scores. This exact identity is the normalization constraint on the slide,

$$\sum_j \mathrm{softmax}(s)_j = \frac{\sum_j e^{s_j}}{\sum_k e^{s_k}} = 1,$$

because numerator and denominator run over the same keys.

The proposed explanation

A query that matches no key strongly must still place its whole weight of one somewhere. A position that every query can see, and whose value changes the output little, could absorb that spare weight, and a position used in this way is a sink. Under the causal mask the first position is visible to every query. Xiao et al. offer this as their explanation of why the initial tokens become sinks (Xiao et al., 2024). It is a hypothesis and has not been proved. The identity singles out no position, so normalization alone does not decide which position becomes the sink.

Week 5 returns to the stability of softmax and of attention logits, and Week 11 to eviction in serving.

Evict the first tokens?

Some heads park weight on the first token.
A rolling cache keeps the last 1,024 tokens.
Once the first ones are evicted, perplexity:
barely changes, or gets much worse?

Answer A3

A learned sink bias is not a retained token

learned no-token sink

$$a_j = \frac{e^{s_j}}{e^{b_h} + \sum_k e^{s_k}}, \qquad a_{\text{sink}} = \frac{e^{b_h}}{e^{b_h} + \sum_k e^{s_k}}$$

- Learned per-head scalar `b_h` (OpenAI, 2025)
- Adds no key, value or position to the cache
- Weights on tokens now sum to less than one

▼ A denominator term gives the spare weight a place without a token

A retained sink is a real position, with a key and a value in the cache. gpt-oss builds a sink into attention instead. It adds a learned scalar b_h, one for each head h, to the softmax denominator (OpenAI, 2025).

Definition. Take a query in head h with score s_j on key j, where j and k run over the keys the query may read. The weight on key j and the share that goes to no token are

$$a_j = \frac{e^{s_j}}{e^{b_h} + \sum_k e^{s_k}}, \qquad a_{\text{sink}} = \frac{e^{b_h}}{e^{b_h} + \sum_k e^{s_k}}.$$

The share a_{sink} belongs to no token, so this is a sink without a token. Equivalently, the a_j and a_{sink} are the softmax of the scores extended by one more entry, b_h, whose weight multiplies no value.

Claim. The token weights sum to $1 - a_{\text{sink}}$, which is below one. The numerators add up to the shared denominator, so

$$\sum_j a_j + a_{\text{sink}} = \frac{\sum_j e^{s_j} + e^{b_h}}{e^{b_h} + \sum_k e^{s_k}} = 1,$$

and therefore $\sum_j a_j = 1 - a_{\text{sink}}$, which is below one because e^{b_h} is positive.

A head that matches no key strongly can now send its spare weight to a_{sink}. No key, value or sequence position belongs to b_h, so it adds nothing to the KV cache and does not change reach. Retained prefix positions are real entries instead and cost cache bytes. gpt-oss, released in 2025, is one model that uses such a term.

An output gate lets a head write almost nothing Read on your own

gated attention output

$$\tilde{o}_{t,h} = o_{t,h} \odot \sigma(W_h^G h_t), \qquad u_t = W_O[\tilde{o}_{t,1}; \ldots ; \tilde{o}_{t,H}]$$

- Per-head sigmoid gate between SDPA and output projection (Qiu et al., 2025)
- Adds nonlinearity and sparse, query-dependent scaling
- Fewer sinks, far fewer loss spikes
- Used in Qwen3-Next (Qiu et al., 2025), A.X K2 (Baek et al., 2026), Arcee Trinity Large (Singh et al., 2026)

▼ A sigmoid gate on each head's output

Under the softmax explanation, a head with nothing to write parks its spare weight on a position whose value changes the output little. Gated attention gives each head a direct way to write little: it multiplies the head's attention output by a sigmoid gate, after scaled dot-product attention (SDPA) and before the output projection (Qiu et al., 2025).

Definition. Let $h_t \in \mathbb{R}^D$ be the normalized hidden state of token t at the input of the attention block, $o_{t,h} \in \mathbb{R}^{D_h}$ the SDPA output of head h at position t (row t of Week 3's context C_h), and $W_h^G \in \mathbb{R}^{D_h \times D}$ the head's learned gate projection. The gated head output and the attention output are

$$\begin{aligned} \tilde{o}_{t,h} &= o_{t,h} \odot \sigma(W_h^G h_t), \\ u_t &= W_O\,[\tilde{o}_{t,1}; \ldots ; \tilde{o}_{t,H}], \end{aligned}$$

where σ is the logistic sigmoid applied to each coordinate, $\odot$ is the elementwise product and W_O is the output projection applied to the stacked head outputs. Each gate value lies between 0 and 1 and scales one coordinate of one head's output, so a value near 0 switches that coordinate off for this token. The paper recommends this elementwise form over a headwise variant with one scalar per head (Qiu et al., 2025). The gate depends on the current token alone, so like the learned sink bias it adds nothing to the KV cache.

Two reasons the gate helps

Qiu et al. give two reasons. The first is nonlinearity. For fixed attention weights, a head's value projection and the output projection merge into one linear map whose rank is at most D_h, because it passes through the head's D_h coordinates. The gate puts a nonlinearity between the two maps, which the authors consider a likely source of its gain. The second is sparsity that depends on the query: the gate is computed from the current query's hidden state, so what it switches off changes from one query to the next, and most gate values are small (Qiu et al., 2025).

What the paper reports

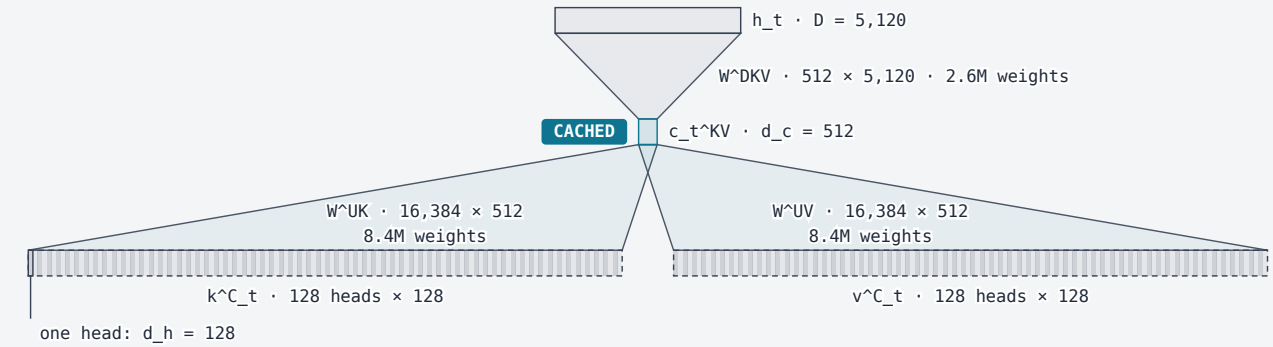
In 15B mixture-of-experts models trained on 400B tokens, the elementwise gate cut the share of attention on the first token, averaged over layers, from 46.7% to 4.8%. In a 1.7B dense model trained on 3.5T tokens, it largely removed the loss spikes. The authors' explanation for the vanishing sink is a hypothesis: the gate can filter out context irrelevant to the current query, so a head with almost nothing to write scales its output toward zero instead of parking weight on the first token. A gate after the value projection reduced the massive activations but not the attention on the first token, so the authors conclude that massive activations are not a necessary condition for attention sinks (Qiu et al., 2025).

Released models use the gate. Qwen3-Next takes the paper's most effective variant, the SDPA output gate (Qiu et al., 2025). A.X K2 applies a gate to the output of each head throughout pre-training (Baek et al., 2026). Arcee Trinity Large gates every coordinate of the attention output before the output projection and names the reduction of loss spikes as a key reason for adopting it (Singh et al., 2026). Week 10's Gated DeltaNet also uses a gate, in a different place: it gates a recurrent state instead of an attention output.

Multi-head Latent Attention

33

Keys and values share one down-projection Read on your own



- Cached: one 512-wide latent per token (DeepSeek-AI, 2024)
- Two separate up-projections: keys and values
- Per-head keys and values rebuilt, never stored

▼ One latent feeds the keys and values of every head

DeepSeek-V2 introduced multi-head latent attention (MLA), which compresses the key-value state of each token into one content latent that every head shares (DeepSeek-AI, 2024). We use two of DeepSeek-V2's symbols: d_h for the head width, the course's D_h, and n_h for the number of query heads, the course's H. We write the number of KV heads as n_{kv}, the course's H_{kv} and the exercise's `n_kv_heads`. The rotary width is d_r, as in the exercise, where DeepSeek-V2 writes d_h^R. Vectors are columns, h_t is the hidden state of token t entering the attention layer, of width D, and d_c is the width of the latent. A subscript h on a matrix or a key indexes the head.

MLA rests on a shared latent and per-head up-projections. A down-projection W^{DKV}, a $d_c \times D$ matrix that all heads share, maps the hidden state to the latent, and each head h has two up-projections W_h^{UK} and W_h^{UV}, each a $d_h \times d_c$ matrix, that define its content key and content value:

$$c_t^{KV} = W^{DKV} h_t, \qquad k_{t,h}^C = W_h^{UK} c_t^{KV}, \qquad v_{t,h}^C = W_h^{UV} c_t^{KV}.$$

Stacking the up-projections of all n_h heads, one block of d_h rows per head, gives two matrices W^{UK} and W^{UV}, each $n_h d_h \times d_c$. Applied to the latent, they give the content keys and the content values of all heads at once, which is how DeepSeek-V2 writes the definitions (DeepSeek-AI, 2024):

$$k_t^C = W^{UK} c_t^{KV}, \qquad v_t^C = W^{UV} c_t^{KV}.$$

Each is $n_h d_h$ wide, and head h's key $k_{t,h}^C$ and value $v_{t,h}^C$ are its h-th block of d_h coordinates.

Keys and values share one low-rank compression

Substituting the latent writes the keys of all heads as $k_t^C = W^{UK} W^{DKV} h_t$. The product $W^{UK} W^{DKV}$ is an $n_h d_h \times D$ matrix, the same shape as the key projection of multi-head attention. Every vector it produces passes through the d_c coordinates of the latent, so the product has rank at most d_c. The values pass through the same latent. DeepSeek-V2 calls this "the low-rank joint compression for keys and values" and names it the core of MLA (DeepSeek-AI, 2024). The compression is low-rank because the latent is narrow, and joint because one latent serves both keys and values.

DeepSeek-V2's widths and weights

Two assumptions fix the numbers.

- **A1. DeepSeek-V2's dimensions.** The hidden state has width $D = 5{,}120$, which DeepSeek-V2 writes d, and the model has $n_h = 128$ heads of width $d_h = 128$ and a latent of width $d_c = 512$ (DeepSeek-AI, 2024). The stacked keys and values are therefore $n_h d_h = 16{,}384$ wide, more than three times the hidden width. Unlike the course configuration, DeepSeek-V2 does not set $D = n_h d_h$, so the up-projections widen the latent beyond h_t.
- **A2. One layer, the key-value path only.** The count covers the matrices that turn h_t into the content keys and values of one layer. The query, the output projection and the part of the key that carries position are left out. The comparator is multi-head attention with the same heads, whose key projection W_K and value projection W_V each map h_t directly to $n_h d_h$ coordinates.

Claim. Under A1 and A2, MLA's key-value path has 19,398,656 weights, about 19 million, while each of the comparator's two projections has 83,886,080, about 84 million. A matrix has as many weights as it has rows times columns:

$$
\begin{aligned}
W^{DKV}:\ d_c\, D &= 2{,}621{,}440 && \text{A1}\\
W^{UK},\ W^{UV}:\ n_h d_h\, d_c &= 8{,}388{,}608 \text{ each} && \text{A1}\\
\text{MLA path: } 2{,}621{,}440 + 2 \cdot 8{,}388{,}608 &= 19{,}398{,}656 && \text{A2}\\
W_K,\ W_V:\ n_h d_h\, D &= 83{,}886{,}080 \text{ each} && \text{A1, A2.}
\end{aligned}
$$

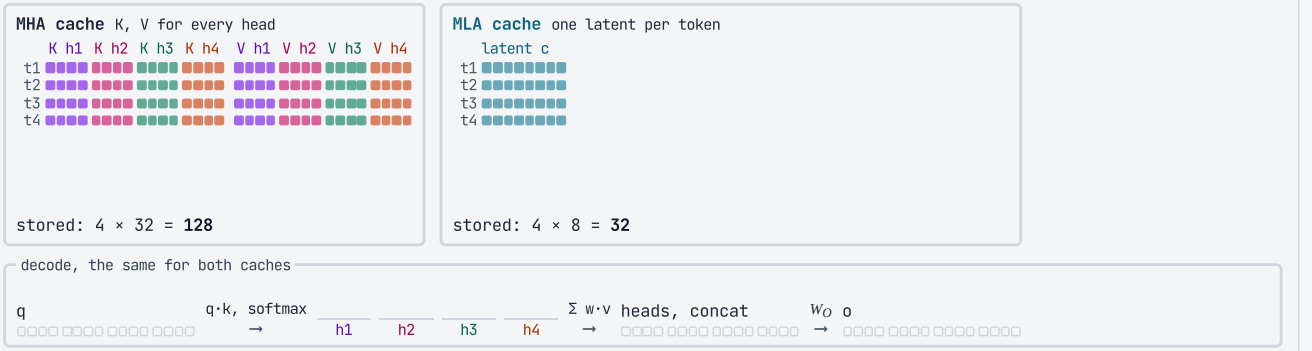
The narrow latent shrinks the weights as well as the cache: in this layer the whole key-value path has fewer than a quarter of the weights of the comparator's key projection alone.

Reading the widget

The widget draws DeepSeek-V2's key-value path for one token in one layer. Every vector is a bar whose length is its width, and all bars share one scale. At the top is the hidden state h_t, 5,120 wide. The funnel below it is W^{DKV}, labeled with its shape, $512 \times 5{,}120$, and its 2.6 million weights. It narrows to the latent c_t^{KV}, 512 wide, drawn in cyan and marked CACHED. Two funnels widen from the latent, W^{UK} on the left and W^{UV} on the right, each labeled $16{,}384 \times 512$ with 8.4 million weights. Both start from the whole latent, because each up-projection reads all 512 coordinates, so their edges cross just below it. They end in the content keys k_t^C and the content values v_t^C, each 16,384 wide and striped into its 128 heads. The first head of the keys is outlined and labeled with its width, $d_h = 128$. Both output bars have dashed outlines because they are rebuilt from the latent whenever attention uses the token and are never stored. The widget shows widths, not the work of the rebuild, and only the content path.

Mechanics: One Shared Latent

ILLUSTRATIVE Prev Next Reset 4 tokens cached, nothing decoding



Toy sizes: model width D = 16, 4 heads of width 4, latent width 8. Each row is one cached token. An MHA row holds a key and a value for every head, 32 numbers grouped by head. An MLA row holds one latent of 8 numbers. MLA runs in the unabsorbed form, as in the exercise: in every decode step it rebuilds each head's keys and values from all cached latents and drops them after use, so the dashed block never counts as stored. Both caches are read in full in every decode step. RoPE is left out here and comes on the next slides. The softmax weights are illustrative.

shared latent and per-head reconstruction

$$c_t^{KV} = W^{DKV} h_t, \qquad k_{t,h}^C = W_h^{UK} c_t^{KV}, \qquad v_{t,h}^C = W_h^{UV} c_t^{KV}$$

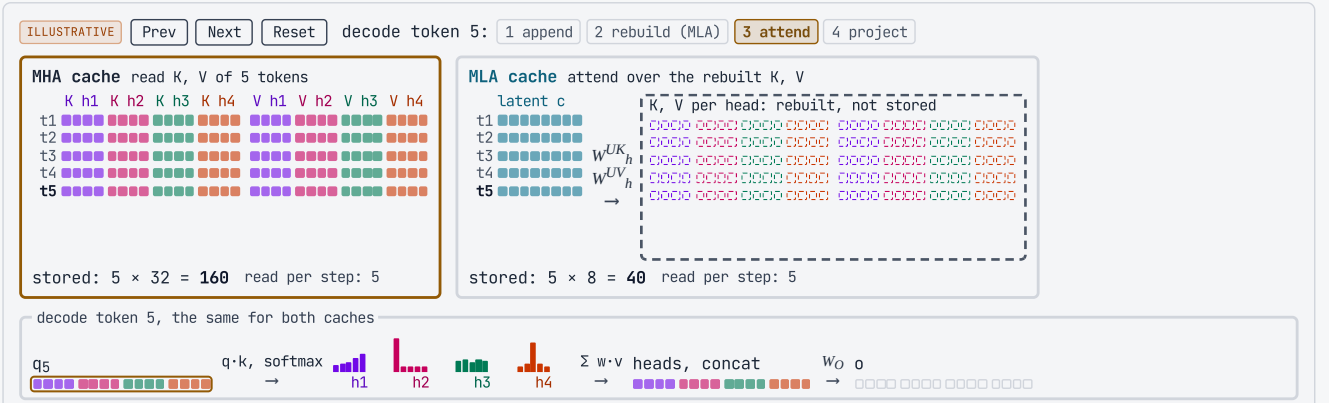
- Cache one latent per token, each head rebuilds K, V (DeepSeek-AI, 2024)

▼ The cache keeps the latent, and each head rebuilds its keys and values

The cache stores the latent of the previous slide, $c_t^{KV} = W^{DKV} h_t$, one per token in each layer. Nothing on the up-projection side is stored: in the unabsorbed form, which the course uses, each head rebuilds its content key $k_{t,h}^C = W_h^{UK} c_t^{KV}$ and content value $v_{t,h}^C = W_h^{UV} c_t^{KV}$ from the latent whenever attention uses the token, and then discards them. The absorbed form folds W_h^{UK} into the query projection and never rebuilds the keys: Claim 1 on the next slide shows the step, and a reading slide after the cache count writes out the whole form. The smaller cache is paid for in work, because every use of a token runs the up-projections again.

What to watch in the stepper

The stepper runs an MHA cache and an MLA cache side by side and decodes one token per four clicks. In the append sub-step, MHA adds a key and a value for every head, while MLA adds one latent. At the stepper's toy sizes that is 32 numbers on the MHA side, a key and a value of width 4 for each of four heads, against 8 on the MLA side. The rebuild sub-step exists only on the MLA side: it applies the up-projections to every cached latent, and the rebuilt keys and values appear in a dashed block that is gone again in the project sub-step, which concatenates the heads' outputs and applies W_O. At the bottom of each cache, the stored count grows by a full MHA row per token on the left and by one latent on the right, so the gap widens with every token. The count read per step is the same on both sides: every cached token, in every step.



Toy sizes: model width D = 16, 4 heads of width 4, latent width 8. Each row is one cached token. An MHA row holds a key and a value for every head, 32 numbers grouped by head. An MLA row holds one latent of 8 numbers. MLA runs in the unabsorbed form, as in the exercise: in every decode step it rebuilds each head's keys and values from all cached latents and drops them after use, so the dashed block never counts as stored. Both caches are read in full in every decode step. RoPE is left out here and comes on the next slides. The softmax weights are illustrative.

Decoding the fifth token, sub-step attend: both caches hold five entries and both are read in full. MLA's dashed block holds the keys and values rebuilt from its five cached latents, which are not stored and are dropped after the step. The bars give each head's softmax weights over the five tokens, and the stored counts read 160 numbers for MHA against 40 for MLA.

The RoPE Conflict: Rotation and Up-Projection Do Not Commute

non-commutation

$$\mathrm{RoPE}_t(W_h^{UK} c_t^{KV}) \neq W_h^{UK}\,\mathrm{RoPE}_t(c_t^{KV})$$

- RoPE depends on position `t`
- Rotating the latent ≠ rotating each key
- So position needs its own branch (DeepSeek-AI, 2024)

▼ A rotation on the content path blocks the absorbed form

RoPE, from Week 3, encodes position by rotation. At position t it turns each pair of coordinates of a query or a key by an angle proportional to t, each pair at its own frequency. Write R_t for this rotation of a whole vector, so that $\mathrm{RoPE}_t(x) = R_t x$. Where the width matters, a superscript names it: $R_t^{(d_h)}$ rotates a key of width d_h and $R_t^{(d_c)}$ a latent of width d_c.

The question is where position can enter MLA's content path. The answer turns on how DeepSeek-V2 runs MLA at inference: it scores queries directly against the cached latent and never rebuilds the per-head keys. Take the content key $k_{t,h}^C = W_h^{UK} c_t^{KV}$ of the previous slide, and the content query $q_{i,h}^C = W_h^Q h_i$ of a query at position $i \geq t$, where W_h^Q is the head's query projection as in Week 3, fixed after training.

Without rotation, the up-projection moves to the query

Claim 1. The content score of query i against token t can be computed from the cached latent, with no key rebuilt. Substituting the key and transposing a product, $(AB)^\top = B^\top A^\top$, moves the up-projection to the query:

$$\begin{aligned}(q_{i,h}^C)^\top k_{t,h}^C &= (q_{i,h}^C)^\top W_h^{UK}\, c_t^{KV} && \text{definition of } k_{t,h}^C \\ &= \left((W_h^{UK})^\top q_{i,h}^C\right)^\top c_t^{KV} && \text{transpose of a product.}\end{aligned}$$

The vector $(W_h^{UK})^\top q_{i,h}^C$ has width d_c and depends on the query alone, so it is computed once per query and head. It equals $(W_h^{UK})^\top W_h^Q h_i$, and the product $(W_h^{UK})^\top W_h^Q$ can be formed once after training. This merge is called absorbing W_h^{UK} into the query projection, and DeepSeek-V2 uses it at inference (DeepSeek-AI, 2024).

A rotation between the two matrices breaks the merge

Claim 2. If RoPE rotated the content query and the content key, the vector that multiplies c_t^{KV} would depend on the key's position t, and no fixed matrix could be merged into the query projection. With all rotations of width d_h and Week 3's relative-score identity $R_i^\top R_t = R_{t-i}$,

$$\begin{aligned}(R_i q_{i,h}^C)^\top (R_t k_{t,h}^C) &= (q_{i,h}^C)^\top R_i^\top R_t\, W_h^{UK} c_t^{KV} && \text{definition of } k_{t,h}^C \\ &= (q_{i,h}^C)^\top R_{t-i}\, W_h^{UK} c_t^{KV} && \text{relative-score identity} \\ &= \left((W_h^{UK})^\top R_{t-i}^\top q_{i,h}^C\right)^\top c_t^{KV} && \text{transpose of a product.}\end{aligned}$$

The rotation R_{t-i} now stands between the query and W_h^{UK}, so the vector against which the latent is scored changes with every cached token t. Computing it takes one product with $(W_h^{UK})^\top$ for every pair of query and key, as much work as rebuilding each key (DeepSeek-AI, 2024).

Rotating the latent instead does not help

A cached rotated latent would produce the rotated key only if $R_t^{(d_h)} W_h^{UK} = W_h^{UK} R_t^{(d_c)}$ held at every position t.

Claim 3. In general this equality fails, so

$$\mathrm{RoPE}_t(W_h^{UK} c_t^{KV}) \neq W_h^{UK}\,\mathrm{RoPE}_t(c_t^{KV}).$$

The two rotations act on different spaces, the d_h coordinates of a key and the d_c coordinates of the latent, and the learned map W_h^{UK} between them has no reason to carry one onto the other. A two-dimensional example with illustrative values shows the failure. Take $d_c = d_h = 2$, the projection $W = \mathrm{diag}(1, 0)$, a rotation R by a quarter turn and the vector $c = (1, 0)^\top$. Projecting first and rotating second gives the first line below, and rotating first gives the second:

$$RWc = R\begin{pmatrix}1\\0\end{pmatrix} = \begin{pmatrix}0\\1\end{pmatrix},$$
$$WRc = W\begin{pmatrix}0\\1\end{pmatrix} = \begin{pmatrix}0\\0\end{pmatrix}.$$

Rotation and up-projection do not commute, so position cannot be folded into the cached latent.

Position gets its own branch

In the unabsorbed form you could still rotate each rebuilt key. DeepSeek-V2's absorbed inference never rebuilds the keys (Claim 1), and a rotation on the content key would block that merge (Claim 2). So it gives position its own branch: the content path stays free of rotation, and beside the content latent the cache holds one rotary key shared by all heads (DeepSeek-AI, 2024).

Decoupled RoPE adds one shared rotary key

unabsorbed MLA QK paths

$$q_{t,h} = [q^C_{t,h}; \mathrm{RoPE}_t(W^{QR}_h h_t)]$$
$$k^R_t = \mathrm{RoPE}_t(W^{KR} h_t)$$

- Rotary key: `d_r` wide, shared by all heads
- Query: content part plus rotary part
- `d_r` is even because RoPE rotates coordinate pairs

▼ The query and the key each gain a rotary part

Position enters MLA through a separate rotary branch. The rotary key projection W^{KR}, a $d_r \times D$ matrix, maps the hidden state to d_r coordinates, and RoPE rotates them into the rotary key k^R_t. Each token has one such key, shared by all heads and cached once per token beside the latent. The query of head h has a content part $q^C_{t,h}$ of width d_h, projected from h_t as in Week 3, and a rotary part of width d_r from the head's own projection W^{QR}_h, a $d_r \times D$ matrix. The query concatenates the two, content part first, and the key of head h concatenates its content key with the shared rotary key:

$$k^R_t = \mathrm{RoPE}_t(W^{KR} h_t),$$
$$q_{t,h} = \left[\, q^C_{t,h};\ \mathrm{RoPE}_t(W^{QR}_h h_t) \,\right],$$
$$k_{t,h} = \left[\, k^C_{t,h};\ k^R_t \,\right].$$

Query and key are both $d_h + d_r$ wide, and the value stays the content value $v^C_{t,h}$, of width d_h. A dot product of two concatenated vectors is the sum of the dot products of their parts, so the unscaled score of a query at position i against a token at position t splits in two:

$$q^\top_{i,h} k_{t,h} = (q^C_{i,h})^\top k^C_{t,h} + \mathrm{RoPE}_i(W^{QR}_h h_i)^\top k^R_t.$$

The first term carries no rotation, so Claim 1 of the previous note applies, and it can be scored against the cached latent. The second carries the position and uses the cached rotary key directly.

The query branches belong to the current query and are not cached. The rotary width d_r must be even, because RoPE rotates coordinates in pairs, as in Week 3.

DeepSeek-V2 also compresses the query, a step the course leaves out. A down-projection W^{DQ} maps h_t to a query latent c^Q_t of width d'_c, 1,536 in DeepSeek-V2, and both query parts come from this latent. The content queries of all heads are $W^{UQ} c^Q_t$, and the rotary queries are $\mathrm{RoPE}_t(W^{QR} c^Q_t)$, where one stacked matrix W^{QR} yields every head's rotary query (DeepSeek-AI, 2024). The query latent is not cached, so the KV cache is the same with or without it. DeepSeek-V2 compresses the query to reduce activation memory during training (DeepSeek-AI, 2024). The bottleneck also cuts weights and work. With DeepSeek-V2's $D = 5{,}120$, one projection from h_t to the queries of all heads has about 126 million weights, and the two projections through c^Q_t have about 46 million together. Each token's query therefore takes roughly a third of the multiply-adds. DeepSeek-V2-Lite, the smaller model in the same report, does not compress the query (DeepSeek-AI, 2024). The course follows that form: the exercise's `q_proj` and `q_rot_proj` project both query parts from h_t directly.

Per layer, MHA or GQA stores a key and a value of width d_h for each of its n_{kv} KV heads, and this MLA layout stores the latent and the shared rotary key:

$$\text{MHA or GQA:}\quad 2\, n_{kv}\, d_h,$$
$$\text{MLA:}\quad d_c + d_r.$$

Full MHA is the case $n_{kv} = n_h$, one KV head for every query head.

How many numbers per token?

Full MHA here caches 32,768 per layer.
MLA caches a latent and one rotary key.
About 8,000, 2,000, 600 or 60?

Answer A4

Absorbed MLA runs as multi-query attention over the latent Read on your own

absorbed MLA score and output

$$s_{i,h,t} = \left[(W_h^{UK})^\top q_{i,h}^C;\ q_{i,h}^R\right]^\top \left[c_t^{KV};\ k_t^R\right]$$
$$u_i = \sum_h \left(W_{O,h} W_h^{UV}\right) \sum_t a_{i,h,t}\, c_t^{KV}$$

- W^{UK} folds into the query, W^{UV} into W_O (DeepSeek-AI, 2024)
- All heads read one 576-wide entry
- Absorbed costs less in decode, unabsorbed in prefill

▼ Absorbing both up-projections leaves one shared entry for all heads

Claim 1 of the RoPE-conflict note showed that a head's content score needs no rebuilt key: query i scores token t as $\left((W_h^{UK})^\top q_{i,h}^C\right)^\top c_t^{KV}$, with a d_c-wide vector that depends on the query alone. The decoupled query adds a rotary part. Write $q_{i,h}^R = \mathrm{RoPE}_i(W_h^{QR} h_i)$ for head h's rotary query, the second part of $q_{i,h}$ in the decoupled-RoPE note. Its score term is $(q_{i,h}^R)^\top k_t^R$, against the cached rotary key. The two terms add up to one dot product of two concatenated vectors, the unscaled score

$$s_{i,h,t} = q_{i,h}^\top k_{t,h} = \left[(W_h^{UK})^\top q_{i,h}^C;\ q_{i,h}^R\right]^\top \left[c_t^{KV};\ k_t^R\right].$$

The left vector is the absorbed query. It is $d_c + d_r = 576$ wide with DeepSeek-V2's dimensions and is computed once per query and head. The right vector is the cache entry of token t, also 576 wide, as the cache stores it. No key is rebuilt.

The value side folds into the output projection

Without absorption, head h's output is the weighted sum of its rebuilt values, where $a_{i,h,t}$ is head h's softmax weight of query i on token t, over the tokens $t \le i$ that the query sees. The up-projection W_h^{UV} is linear, so it moves out of the sum:

$$\begin{aligned} o_{i,h} &= \sum_t a_{i,h,t}\, v_{t,h}^C && \text{weighted sum of values} \\ &= \sum_t a_{i,h,t}\, W_h^{UV} c_t^{KV} && \text{definition of } v_{t,h}^C \\ &= W_h^{UV} \sum_t a_{i,h,t}\, c_t^{KV} && \text{linearity.} \end{aligned}$$

The softmax weights now average 512-wide latents, and the up-projection runs once per query and head instead of once per cached token. As in the output-gate note, the output projection W_O maps the stacked head outputs to $u_i = W_O[o_{i,1}; \dots ; o_{i,n_h}]$. Split it into blocks, $W_O = [W_{O,1} \ \cdots \ W_{O,n_h}]$, where block $W_{O,h}$ holds the d_h columns that multiply head h's output. Then

$$u_i = \sum_h W_{O,h}\, o_{i,h} = \sum_h \left(W_{O,h} W_h^{UV}\right) \sum_t a_{i,h,t}\, c_t^{KV},$$

and each $W_{O,h} W_h^{UV}$ is one $D \times d_c$ matrix that can be formed once after training. Merging is optional. An implementation can also apply W_h^{UV} to the averaged latent and then $W_{O,h}$, which gives the same output for $d_h d_c + D d_h$ multiply-adds per head instead of $D d_c$. DeepSeek-V3.2's figure of the MQA mode draws W^{UV} as such a separate step (DeepSeek-AI, 2025). DeepSeek-V2 states both merges: "since W^{UK} can be absorbed into W^Q, and W^{UV} can be absorbed into W^O, we even do not need to compute keys and values out for attention" (DeepSeek-AI, 2024). Its W^Q and W^O are the query projection and the output projection.

All heads read one shared entry

In the absorbed form, every head scores the same cache entry $[c_t^{KV}; k_t^R]$ and averages the same latents c_t^{KV}. Only the queries and the merged output matrices differ between heads. This is Week 3's multi-query attention (MQA), in which every query head reads one shared key and value head. Here the shared key head is $d_c + d_r = 576$ wide and the shared value head $d_c = 512$ wide, for $n_h = 128$ query heads. We derive these two widths from DeepSeek-V2's dimensions. The value head is the first 512 coordinates of the key head, the latent itself, so the cache still holds 576 numbers per token, not 576 + 512.

Absorption changes how the score is computed, not its value. The softmax therefore keeps DeepSeek-V2's scale, which is set by the width of the unabsorbed query and key (DeepSeek-AI, 2024):

$$a_{i,h,t} = \mathrm{softmax}_t\left(\frac{s_{i,h,t}}{\sqrt{d_h + d_r}}\right).$$

The absorbed dot product runs over 576 coordinates, but the divisor stays $\sqrt{d_h + d_r} = \sqrt{192}$.

When each form is cheaper

Absorption moves work from every cached token to every query. Whether that pays depends on how many queries use each cached token, which a count of multiply-adds shows.

- **A1. DeepSeek-V2's dimensions.** $d_h = 128$, $d_c = 512$ and $d_r = 64$ (DeepSeek-AI, 2024).
- **A2. Multiply-adds only.** A matrix-vector product costs one multiply-add per weight, and a dot product or a weighted sum one per coordinate. The softmax is left out.
- **A3. Unabsorbed decode rebuilds at every step.** In the unabsorbed form, every decode step rebuilds the content key and value of every cached token, as the stepper showed. Keeping them would bring back the full MHA cache.
- **A4. Prefill shares the rebuild.** In prefill, and in training, which processes all positions at once in the same way, each token's key and value are rebuilt once and used by every query that sees the token, so the count per pair leaves the rebuild out, as it leaves out the absorbed form's work per query.

Claim. Under A1 to A3, a decode step costs 131,392 multiply-adds per cached token and per head in the unabsorbed form and 1,088 in the absorbed form, plus a cost per query that does not grow with the context. Under A1, A2 and A4, one query-key pair costs 320 multiply-adds per head in prefill in the unabsorbed form and 1,088 in the absorbed form.

In decode, the unabsorbed form rebuilds the content key and the content value, $d_h d_c$ multiply-adds each, then scores the $d_h + d_r$ coordinates of the key and adds the d_h coordinates of the value. The absorbed form scores the 576-wide entry and adds the 512-wide latent:

$$\begin{aligned}
\text{unabsorbed} &= 2\,d_h d_c + (d_h + d_r) + d_h && \text{A2, A3} \\
&= 131{,}072 + 192 + 128 = 131{,}392 && \text{A1} \\
\text{absorbed} &= (d_c + d_r) + d_c && \text{A2} \\
&= 576 + 512 = 1{,}088 && \text{A1.}
\end{aligned}$$

Per query and head, the absorbed form also computes $(W_h^{UK})^\top q_{i,h}^C$, $d_h d_c = 65{,}536$ multiply-adds, and applies W_h^{UV} once to the averaged latent, another 65,536. That cost is the same at any context length. In prefill the rebuild drops out of the count per pair and head:

$$\begin{aligned}
\text{unabsorbed} &= (d_h + d_r) + d_h = 192 + 128 = 320 && \text{A1, A2, A4} \\
\text{absorbed} &= (d_c + d_r) + d_c = 576 + 512 = 1{,}088 && \text{A1, A2.}
\end{aligned}$$

In decode, one query meets every cached token, and the unabsorbed form pays the rebuild for each of them at every step, about 120 times the absorbed form's work per token. From two cached tokens on, that saving outweighs the absorbed form's 131,072 multiply-adds per query and head, so decode favors the absorbed form. In prefill and training the rebuild is shared, and the unabsorbed form's narrower vectors make each pair 3.4 times cheaper, so prefill and training favor the unabsorbed form. These are counts of arithmetic, not timings.

What DeepSeek reports

DeepSeek-V3.2's paper calls the two forms the MHA mode and the MQA mode of MLA. The caption of its figure on the two modes reports how DeepSeek-V3.1-Terminus, the model that DeepSeek-V3.2 continues training from, uses them: "For DeepSeek-V3.1-Terminus, the MHA mode is used for training and prefilling, while the MQA mode is used for decoding" (DeepSeek-AI, 2025). That split agrees with the counts above, which are this lecture's own estimate. DeepSeek-V3.2 implements its sparse attention "based on the MQA mode of MLA", where each latent entry is shared across all query heads of a query token (DeepSeek-AI, 2025).

What the course uses

The course and the exercise use the unabsorbed form, which keeps every stored tensor and every rebuild explicit. Both forms implement the same model, and which one runs faster has to be measured. Absorbed kernels are optional material outside the core weeks. Week 11 covers what MLA means for serving: cache capacity and serving parallelism.

39

Store less per token, or read fewer tokens Read on your own

stored per token	read per query
GQA: fewer KV heads	Learned sparsity: which tokens are read
MLA: one latent plus rotary key	

▼ Stored state and entries read are separate counts

Two counts describe what attending over a long history costs, and a mechanism can change one of them without changing the other. The first is what the cache stores for each token in each layer. The second is how many of the stored entries one query reads and attends to.

GQA and MLA change the first count. GQA stores fewer KV heads. MLA stores the content latent c_t^{KV} and the shared rotary key k_t^R in place of a key and a value for every head. Learned sparse attention changes the second count: it chooses which stored tokens or blocks each query reads and attends to. Dense MLA still reads every valid position, as the stepper's read counter showed. Which entries stay in the cache is a separate decision from which entries a query reads.

46

DeepSeek-V3.2 selects from the MLA cache Read on your own

- Origin: DeepSeek-V2 (2024) (DeepSeek-AI, 2024)
- DeepSeek-V3.2: sparse attention over MLA (DeepSeek-AI, 2025)
- Next: choose which tokens to read

▼ From a smaller cache to choosing what to read

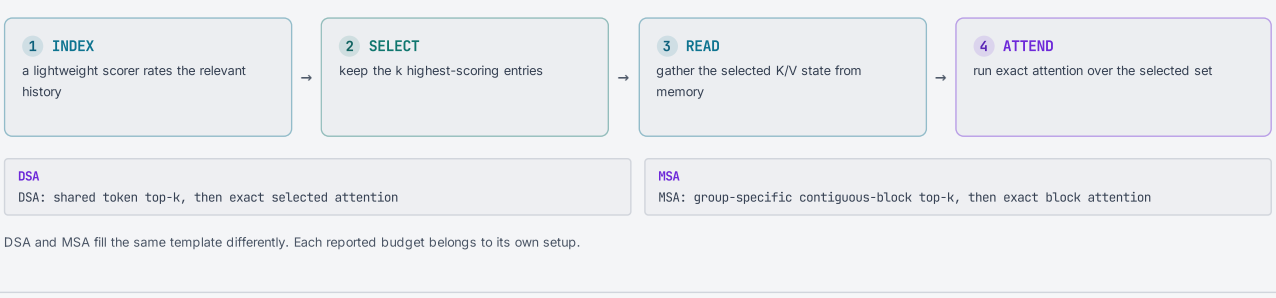
MLA comes from DeepSeek-V2, published in 2024, which introduced the reference formulation this course uses (DeepSeek-AI, 2024). A later model in the same line, DeepSeek-V3.2, builds on it. Its trainable sparse attention selects, from the MLA representation, which entries each query reads (DeepSeek-AI, 2025). That leaves the question of how a trainable mechanism chooses what each query reads.

Trainable Sparsity

41

Learned sparsity chooses what to read

- Window: keys chosen by position
- Learned sparsity: keys chosen per query
- Selecting fewer keys does not shrink the cache



DSA and MSA fill the same template differently. Each reported budget belongs to its own setup.

▼ Four stages, each with its own cost

A sliding window decides by position which keys a query reads: the w most recent positions, fixed before the query's content is known. Learned sparse attention decides per query. A trained scorer rates the stored history for the current query, and the query reads only the entries that score highest. Both read fewer keys than dense attention.

The template

The learned designs of this tab share one template of four stages. Write k_{sel} for the number of entries selected per query, the k of top-k, with a subscript that keeps it apart from a key.

Definition. For the query at position t, learned sparse attention runs four stages:

1. **Index.** A lightweight scorer rates every entry of the stored history that the query may see.
2. **Select.** The k_{sel} highest-scoring entries are kept. An entry is a token or a contiguous block of tokens. Write $\mathcal{I}_t$ for the set of tokens the kept entries cover.
3. **Read.** The keys and values of the tokens in $\mathcal{I}_t$ are gathered from the stored cache.
4. **Attend.** Exact softmax attention runs over $\mathcal{I}_t$ only. For one head, with s_j the attention score of the query against key j and v_j the value of token j, the output is

$$o_t = \sum_{j \in \mathcal{I}_t} \frac{e^{s_j}}{\sum_{j' \in \mathcal{I}_t} e^{s_{j'}}} v_j.$$

The softmax is exact over the selected set: its denominator sums over $\mathcal{I}_t$, and a token outside $\mathcal{I}_t$ adds nothing to o_t.

Two designs fill in the selection differently. DeepSeek sparse attention (DSA) selects tokens, with one top-k set for all heads. MiniMax sparse attention (MSA) selects contiguous blocks, separately for each GQA group.

The selected count k_{sel} is the number of entries that the read and attend stages handle for one query. It is not the size of the stored cache: selection leaves every token's state in the cache and decides only which stored entries a query reads. Nor does k_{sel} give the latency, because the index and select stages do work of their own.

The cost of each stage

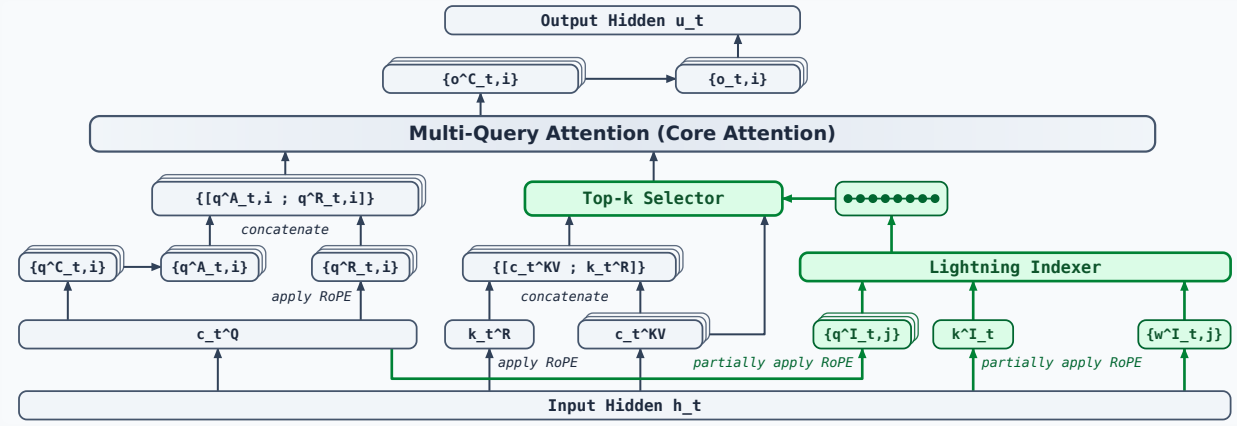
- **Index.** The scorer costs index time. Its quality is the **selection recall**: the share of the entries the query needs that the selection keeps.
- **Select.** The selector emits token or block addresses, one list for all heads or one per group, at the cost of selection time, synchronization and address metadata.
- **Read.** The gather costs the bytes requested, the bytes transferred and the gather time.
- **Attend.** Exact attention over the selected set costs exact-attention time.

These measurements locate different bottlenecks, so report selection recall, bytes read and synchronized latency separately. Only the synchronized end-to-end latency describes the whole pipeline.

StepFun reports one trade-off between index cost and selection quality for its Step 5 Preview model. An indexer that scores blocks of 8 tokens and keeps 4,096 tokens reaches a token coverage of 78.06%, against 79.09% for single tokens with 2,048 kept, and it cuts the cost of indexing and top-k selection by a factor of 8 (StepFun Team, 2026). The post reports token coverage without defining it.

DSA scores tokens, then selects a shared set Read on your own

- DeepSeek sparse attention (DSA), V3.2 (DeepSeek-AI, 2025)
- Lightning indexer scores every token
- Top-k: one shared set, `k = 2,048`



Adapted from DeepSeek-AI, 2025

The diagram follows Figure 2 and §2.1 of the DeepSeek-AI report, in which DSA is built on MLA. The green path is DSA's indexer and selection path. Core attention then uses the selected latent key-value entries, which all heads share.

▼ What top-k selection saves, and what it leaves out

DeepSeek sparse attention (DSA) is the sparse attention of DeepSeek-V3.2, built on the MLA cache (DeepSeek-AI, 2025). It fills the four-stage template with tokens. A lightning indexer scores every causally visible token for the current query, and a top-k selector keeps the k_{sel} highest-scoring tokens. DeepSeek-V3.2 implements DSA on MLA's MQA mode, the absorbed form of the MLA tab, where all query heads read the same cache entries, so one selected set serves all heads. Core attention then runs exact attention over the selected latent entries. The MLA cache still holds the latent and the rotary key of every earlier token, and the top-k decides which of them each query reads. The reported setup uses $k_{\text{sel}} = 2{,}048$, the slide's k. That is one configuration, and it does not define DSA.

Reading the figure

`DsaMlaArchitecture` is adapted from the paper's figure and shows two paths into the core attention block. On the left, the query path forms each head's content and rotary queries and concatenates them. The label c_t^Q is DeepSeek-V2's query latent, which the decoupled-RoPE note describes and the course leaves out. The figure keeps DeepSeek's indices, t for the token and i for the head. Its $q_{t,i}^A$ is the absorbed content query of the MLA tab, $(W_h^{UK})^\top q_{t,h}^C$ for head $h = i$. DeepSeek-V3.2's appendix figure on MLA's MQA mode labels this step $W_i^{UK} q_{t,i}^C$, without the transpose that the shapes require (DeepSeek-AI, 2025). Read $[q^A; q^R]$ as the absorbed query, 576 wide with DeepSeek-V2's dimensions, scored against the cached entry $[c_t^{KV}; k_t^R]$. On the output side, $o_{t,i}^C$ is head i's weighted average of the selected latents, and W^{UV} turns it into the head output $o_{t,i}$ before the output projection forms u_t: the value side of the absorbed form, applied in two steps. In the middle, the MLA latent c_t^{KV} and the shared rotary key k_t^R reach the Top-k Selector. The indexer's branches, drawn in green, feed the Lightning Indexer, whose scores the drawing shows only as a row of identical circles. The selector chooses one shared set of latent entries, which is gathered from memory before the core multi-query attention applies exact attention.

What selection saves in core attention

Two counts show what selection saves in core attention, for one decode step and for a whole causal prefill. Both count the query-token pairs that core attention scores and weights.

- **A1. Causal top-k selection.** Number the queries $t = 1, \ldots, S$, where S is the context length. The query at position t sees the t tokens up to and including itself, and the selector keeps at most k_{sel} of them, so the query attends to $\min(t, k_{\text{sel}})$ tokens.
- **A2. The example sizes.** Take an example context of $S = 131{,}072$ tokens and the reported budget $k_{\text{sel}} = 2{,}048$.

Claim 1. Under A1 and A2, a decode step at position S attends to 1.5625% of the cached tokens.

A1 gives $\min(S, k_{\text{sel}}) = k_{\text{sel}}$ tokens out of S, so the share is

$$\frac{k_{\text{sel}}}{S} = \frac{2{,}048}{131{,}072} = 0.015625.$$

Claim 2. Under A1, causal prefill attends to

$$\sum_{t=1}^{S} \min(t, k_{\text{sel}}) = S\,k_{\text{sel}} - \frac{k_{\text{sel}}(k_{\text{sel}}-1)}{2} \tag{1}$$

query-token pairs, at most $S\,k_{\text{sel}}$. Under A1 and A2, that is about 3.10% of the pairs of dense causal attention.

Split the sum where $\min(t, k_{\text{sel}})$ stops growing:

$$\begin{aligned}
\sum_{t=1}^{S} \min(t, k_{\text{sel}}) &= \sum_{t=1}^{k_{\text{sel}}} t + \sum_{t=k_{\text{sel}}+1}^{S} k_{\text{sel}} && \text{by A1} \\
&= \frac{k_{\text{sel}}(k_{\text{sel}}+1)}{2} + (S - k_{\text{sel}})\,k_{\text{sel}} && \text{Gauss sum} \\
&= S\,k_{\text{sel}} - \frac{k_{\text{sel}}(k_{\text{sel}}-1)}{2} && \text{algebra}
\end{aligned}$$

Dense causal attention has $\sum_{t=1}^{S} t = S(S+1)/2$ pairs. With the sizes of A2,

$$\begin{aligned}
\frac{k_{\text{sel}}(k_{\text{sel}}+1)}{2} &= \frac{2{,}048 \times 2{,}049}{2} \\
&= 2{,}098{,}176 && \text{first } k_{\text{sel}} \text{ queries} \\
(S - k_{\text{sel}})\,k_{\text{sel}} &= 129{,}024 \times 2{,}048 \\
&= 264{,}241{,}152 && \text{other queries} \\
\text{selected pairs} &= 266{,}339{,}328 && \text{by (1)} \\
\frac{S(S+1)}{2} &= 8{,}590{,}000{,}128 && \text{dense pairs} \\
\text{share kept} &= 0.0310057\ldots
\end{aligned}$$

Equation (1) is the selected-attention term, the $O(S\,k_{\text{sel}})$ cost of core attention under DSA. At a fixed k_{sel} it grows linearly in S, where the dense count $S(S+1)/2$ grows with S^2.

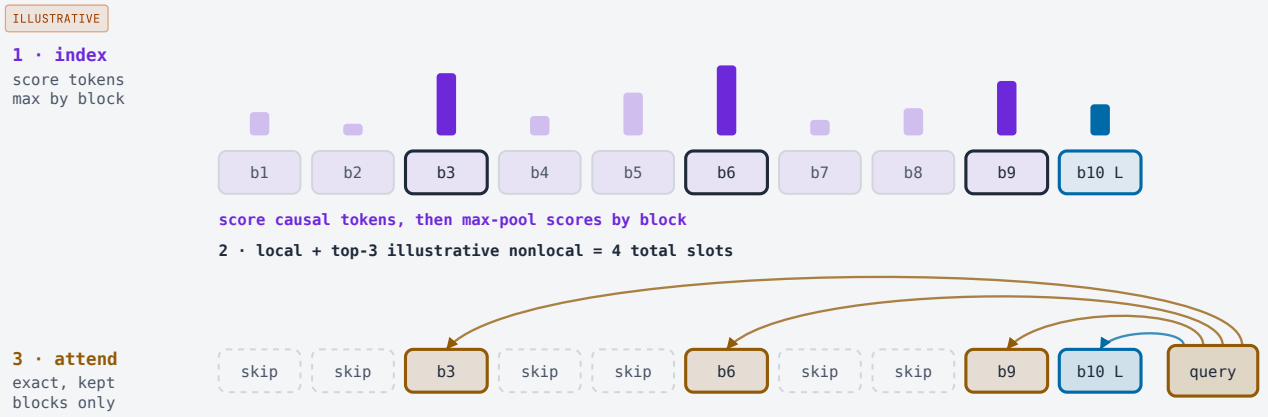
What the counts leave out

The term counts core attention only, without the indexer, the selection, the gathers and their synchronization, so neither fraction predicts an end-to-end speedup.

DSA follows the method lineage of Native Sparse Attention (NSA), an earlier natively trainable sparse attention design (Yuan et al., 2025).

MSA selects blocks, not tokens Read on your own

- MiniMax sparse attention (MSA) (Lai et al., 2026)
- Token scores max-pooled over blocks of 128 tokens
- Per GQA group: 16 blocks, local included



The figure shows MSA from one query's view. The index branch scores the causally visible tokens and max-pools those scores within each 128-token block. The local block always takes one slot, and the three highest nonlocal maxima fill the other slots shown. Each GQA group selects its own contiguous blocks. The main branch then runs exact attention over the causally visible tokens in the selected blocks only.

▼ Each group reads 16 blocks of 128 tokens

MiniMax sparse attention (MSA) follows the same template, with contiguous blocks as the unit of selection (Lai et al., 2026). MiniMax has released a model built on it, MiniMax M3 (Lai et al., 2026).

The paper reports one setup for its research models. The history is divided into blocks of 128 tokens. The index branch scores every causally visible token for the current query and then max-pools these token scores within each block, so a block's score is the highest score of any visible token in it. Selection happens once per GQA group, the query heads that share one KV head (Week 3), and each group keeps 16 blocks. The local block, the one that contains the query, is always among them. The other 15 slots go to the blocks with the highest scores. These 15 blocks can lie anywhere in the visible history, so block selection does not shrink the reach to a fixed local window. Exact attention then runs over the causally visible tokens in the selected blocks.

Claim. In this setup, one GQA group attends to at most 2,048 tokens per query:

$$|\mathcal{I}_t| \le 16 \times 128 = 2{,}048,$$

where $\mathcal{I}_t$ is the set of tokens the group's selected blocks cover for the query at position t.

Each of the 16 selected blocks holds at most 128 causally visible tokens, and the local block counts as one of the 16, not as a seventeenth.

`MsaBlockSparse` follows one query over ten blocks, b1 to b10, each bar a block's pooled score. The local block b10, marked L, is kept whatever its score, the three highest bars fill the other slots, and the blocks marked skip are not read. The display keeps four slots in place of the reported 16, and its scores are illustrative.

Tokens and blocks trade precision for contiguity Read on your own

tokens	blocks
Whole budget on chosen tokens	Contiguous reads
Scattered K/V reads	Unchosen neighbors read too

▼ Following the selected addresses into memory

Token selection and block selection spend a budget differently, and the difference shows in memory.

Token selection spends the whole budget on entries the indexer chose. The selected tokens are usually not consecutive, though, so in a row-major layout of the keys and values their addresses are not adjacent. The gather then issues separate memory transactions, or separate cache-line fetches, wherever two addresses do not share one.

Block selection reads contiguous ranges. With blocks of n_b tokens, a selected block r expands to the token range

$$[\, r\, n_b,\ (r+1)\, n_b\,),$$

which can be fetched in coalesced transactions, in which neighboring threads read neighboring addresses in one transfer. In exchange, every token of the block arrives, including neighbors that the indexer did not choose on their own. Reading them is called overfetch.

A second choice is how widely one selection is shared. A selection shared by all heads amortizes the decision: one top-k and one address list serve every head. Selections that differ per group keep more diversity, since each group can read different history, at extra index and gather cost. When groups select different address lists, the metadata must be stored or communicated per group, and the groups must synchronize before the exact-attention result is complete.

Any indexer can miss evidence. A token outside the selected set cannot influence the query, however much it mattered, because the softmax of exact attention runs over the selected set alone. That is a question of quality, which is why selection recall is measured apart from the bytes read and the synchronized latency. Neither address pattern wins in general. The layout, the kernel, the hardware, the sequence shape and the distribution of queries decide the trade-off, so measure the end-to-end speed.

DSA and MSA fill one template differently Read on your own

- DSA: indexer scores tokens, shared top-2,048 (DeepSeek-AI, 2025)
- MSA: 128-token blocks, 16 per GQA group (Lai et al., 2026)
- Native Sparse Attention (NSA): the lineage (Yuan et al., 2025)
- Selection budgets hold only for their reported setups

▼ Two designs on one template

DSA and MSA both learn where exact attention should look, and both run the same four stages: index, select, read, and exact attention over the selected set. They differ in what they select, tokens or blocks, and in who shares a selection, all heads or each GQA group. Both descend from Native Sparse Attention (NSA) (Yuan et al., 2025). Week 10 turns to hybrid attention, which interleaves linear and full attention layers.

DeepSeek's global KV cache per token shrank 437-fold since 2023 Read on your own



Redrawn from Figure 1b of the DeepSeek-V4.1-Flash report. The bars share one linear axis, so the last two are slivers. Each bar is a different model in its own deployed storage format.

- Narrower entries in lower precision: MLA, FP8
- Fewer entries along the sequence: tokens merged
- Compressed Sparse Attention (CSA): 4 tokens, one entry (DeepSeek-AI, 2026a)
- Heavily Compressed Attention (HCA): 128 tokens, one entry
- Fewer entries across layers: CSA2 reuses them (DeepSeek-AI, 2026b)

▼ Three ways DeepSeek shrank the cache

Figure 1b of the DeepSeek-V4.1-Flash report plots the global KV cache of four DeepSeek generations in bytes per token, summed over all layers that keep one (DeepSeek-AI, 2026b). The four bars share one linear axis: DeepSeek-V1 (November 2023) at 389,120 bytes, DeepSeek-V3.2 (December 2025) at 48,068, DeepSeek-V4-Flash (April 2026) at 3,514 and DeepSeek-V4.1-Flash (September 2026) at 890. From the first bar to the last the factor is $389{,}120/890 \approx 437.2$, which the report's caption rounds to 437-fold (DeepSeek-AI, 2026b).

The four bars are four different models, each in its own deployed storage format. They differ in size, depth and much else, so the figure is no controlled ablation, and no single step isolates one mechanism. Between the bars, three mechanisms changed what is stored. V3.2 stores a narrow MLA entry, with the latent in FP8, in place of V1's per-head keys and values. V4-Flash keeps fewer entries along the sequence, because CSA and HCA merge tokens. V4.1-Flash keeps fewer entries across layers, because most of its layers reuse the entries of an earlier layer.

Fewer entries along the sequence: CSA and HCA

DeepSeek-V4, the next model after V3.2, keeps DSA's selection and also reduces the number of entries along the sequence (DeepSeek-AI, 2026a). In Compressed Sparse Attention (CSA), a trained compressor merges the key-value entries of every 4 tokens into one entry, and DSA's lightning indexer and top-k select among these compressed entries. Heavily Compressed Attention (HCA) merges every 128 tokens into one entry and attends to every earlier entry, with no indexer. CSA and HCA layers are interleaved. Both add an uncompressed sliding window over the most recent 128 tokens, because a query attends only to earlier compressed entries and could not otherwise reach the earlier tokens of its own, not yet compressed group (DeepSeek-AI, 2026a). In terms of stored state and entries read, compression changes what the cache stores, and CSA's top-k changes which stored entries each query reads.

Fewer entries across layers: CSA2

DeepSeek-V4.1-Flash replaces V4's mix of CSA and HCA with Compressed Sparse Attention 2 (CSA2) alone, which reuses the main KV entries, the indexer keys and the top-k indices across layers (DeepSeek-AI, 2026b). A Full layer computes and stores new entries and selects a fresh top-k. A Reindex layer reuses a preceding layer's entries and indexer keys and selects its own top-k. A Reuse layer reuses both the entries and the top-k indices. Of V4.1-Flash's 40 layers, the first two use only the sliding window and keep no global entries. The other 38 are 4 Full, 4 Reindex and 30 Reuse layers, so 34 layers reuse the entries of 4 (DeepSeek-AI, 2026b). V4.1-Flash also stores its main KV entries in FP4, a 4-bit format with one 8-bit scale for every 16 values, while its sliding-window cache stays in FP8 (DeepSeek-AI, 2026b).

What the figure leaves out

"Global" leaves two caches out: the sliding-window cache that V4 and V4.1 keep beside the compressed entries, and the cache of any multi-token prediction layers.

Summary

47

What limits attention, and how models cut it

1. Roofline finds what binds, MFU compares with a matching peak
2. FlashAttention: same function, fewer bytes, extra FLOPs
3. Window: less reach, savings need skipping and eviction, sinks help
4. Store less (MLA, paid in rebuilds) or read less (DSA, MSA, paid in indexing and recall)
5. Measure the bottleneck: no mechanism wins by name

▼ Each mechanism cuts one cost and pays with another

The question for this lecture was what limits attention on a GPU, arithmetic or bytes, and how modern models cut that cost. Whichever resource binds for the workload limits it, and each mechanism cuts one cost and pays with another. Five quantities keep them apart:

1. the semantic reach, which earlier tokens can influence a query;
2. the cache stored for each token;
3. the bytes read from memory;
4. the index work, spent on choosing what to read;
5. the kernel work, spent on computing attention.

Which resource binds

The roofline compares an operation's arithmetic intensity $I = F_{\text{useful}}/B_{\text{HBM}}$, its useful FLOPs per byte of HBM traffic under a stated traffic model, with the ridge point $I_{\text{ridge}} = P_{\text{peak}}/\beta$. Below the ridge bandwidth binds, and above it compute can. Counting only its score-matrix traffic, naive attention at the lecture's example shape reached about 85 FLOP per byte, against an H100 SXM ridge of about 295. Single-device MFU divides the model's FLOP/s by one peak that matches the run's precision and its dense or sparse convention, $\text{MFU}_{1\text{GPU}} = P_{\text{model}}/P_{\text{peak,matching}}$.

FlashAttention moves fewer bytes

FlashAttention computes the same function as dense attention. Tiling, online softmax and recomputation keep the score matrix out of HBM, at the price of extra arithmetic in the backward pass. For the attention of GPT-2 medium on an A100 at sequence length 1,024, forward and backward pass together, it did 13% more arithmetic and ran three times faster (Dao et al., 2022), because the traffic it avoided cost more time than the arithmetic it added.

A window gives up reach

A local layer reads only the w most recent positions, and depth widens the reach only linearly, to $R_l = 1 + l(w-1)$ positions after l layers, capped by the prefix. The mask changes which keys are visible, which is semantic sparsity. The work falls to $O(Sw)$ only with a kernel that skips the masked keys, which is execution sparsity, and the decode cache falls to $O(w)$ only with eviction that matches the window.

Evicting the first positions from a rolling window made perplexity much worse, and keeping a few restored it. In Xiao et al.'s models these positions are attention sinks, on which some trained heads put much of their weight (Xiao et al., 2024). The recovery is observed, and the softmax explanation is a hypothesis. A retained sink is a real position in the cache, while gpt-oss's learned sink bias is a per-head term in the softmax denominator with no token behind it (OpenAI, 2025).

Storing less and reading less

MLA stores one shared content latent and one shared rotary key per token and pays in reconstruction: in the unabsorbed form each head rebuilds its content key and value at every use. Which form stores less depends on the configuration. With 16 query heads, $n_{kv} = 2$, $d_h = 128$, $d_c = 512$ and $d_r = 64$, MHA stores 4,096 elements per token per layer, GQA 512 and MLA 576, so here GQA stores slightly fewer than MLA. DSA, which selects tokens, and MSA, which selects blocks, leave the stored cache as it is and change which entries each query reads. They pay in index work and in recall, because a token the indexer does not select cannot influence the query.

Modern models combine these mechanisms, and how a combination performs is a measurement of one setup. Measure the bottleneck of your workload before you choose, because no mechanism wins by its name.

Check yourself

1. What is the difference between semantic and execution sparsity?
2. Which peak, bandwidth, dtype, traffic model, model-FLOP convention, and measured rate support a roofline or MFU conclusion?

Work on the reflection questions without hints, and bring your reasoning to the tutorial. Week 5 turns from what attention costs on a GPU to how a model is trained: optimizers, learning-rate schedules, and numerical stability.

48

Choose the mechanism that changes the bottleneck Read on your own

condition	response
If exact dense semantics are required and HBM traffic binds,	use an IO-aware dense kernel.
If bounded streaming memory and local semantics are acceptable,	pair SWA with matching eviction and test retained sinks separately.
If dense reach must remain while per-token stored state falls,	compare GQA and unabsorbed MLA, then measure decode.
If long reach is required but attending every stored token is too costly,	evaluate learned selection recall, gather bytes, and synchronized latency.

- Attribute each change to its mechanism

▼ Each condition points to one mechanism

Each row names a condition of the workload and the mechanism that changes its bottleneck. If exact dense attention is required while HBM traffic binds, an IO-aware dense kernel such as FlashAttention computes the same function with fewer HBM reads and writes. If the cache must stay within a fixed size while text streams in, and nearby tokens suffice, pair the sliding window with matching eviction, and test retained sinks separately, because they change perplexity and cost cache bytes of their own. If dense reach must stay while the state stored per token falls, compare GQA with unabsorbed MLA and measure decoding, where every step reads the cache. If long reach is needed and attending to every stored token costs too much, evaluate learned selection on its recall, the bytes it gathers and its synchronized latency.

When mechanisms are combined, attribute each measured change to the mechanism that caused it. GQA and a window with matching eviction both shrink the decode cache, for example, so in a model that uses both, a smaller cache could come from either.

49

What these mechanisms leave open Read on your own

- Numerical stability of attention
- Multi-GPU throughput and MFU
- Hybrid sparse or linear attention
- Serving: cache allocation, eviction, paging

▼ Four problems remain after the attention rule is chosen

Choosing an attention mechanism leaves four problems open.

1. Numerical stability. Attention logits and softmax must stay stable under mixed precision, for example with qk-norm, which normalizes queries and keys before their dot product, or softcapping, which caps the logits.
2. Throughput on several GPUs, where throughput and MFU are measured across all devices. This lecture computed MFU for one.
3. Hybrid attention, which combines sparse or linear attention beyond this lecture's mechanisms.
4. Serving. A server allocates, evicts and pages the KV cache, as PagedAttention does, and chooses MLA's absorbed or unabsorbed form for each phase.

Exercise: Exact Semantics and Stored State Read on your own

- Window: band, `mask_mod`, `layer_types`
- Sinks: window with and without sink columns, against full context
- MLA: latent plus rotary key, `d_c + d_r`

▼ Three results the exercise asks you to build

The exercise asks for three results, each about what attention computes or what it stores. Its README, `exercises/week-04/README.md`, gives every detail.

The sliding window

You implement the inclusive band

$$\max(0,\, i - w + 1) \le j \le i$$

in `causal_mask` and compose it with the Week 3 rules for causality, padding and packed documents. Test three cases: at $w = 1$ a query reads only its own key, the band clips at key 0 for the early queries, and at $w \ge S$ it must equal plain causal attention. You also state the window as a FlexAttention mask function, `window_mask_mod(window)`, which must agree with `causal_mask` on a batch without padding. The per-layer setting `layer_types` names each layer `sliding_attention` or `full_attention`.

The attention-sink comparison

The course supplies the trace, the model and a fixed evaluation slice of text. You compute three perplexities on that slice:

1. under the window alone;
2. under the same window widened, within the causal band, by the sink columns the trace names;
3. under the plain causal mask with no window.

The third is the full-context reference against which both windowed numbers are read. The text, tokenizer, model, slice, scoring positions and seed stay fixed, and the window and the sink positions come from the trace, so only the retained positions differ between the two windowed numbers. The exercise trains no sink and runs no eviction loop. Each perplexity comes from one forward pass under a mask, so neither windowed number describes a serving policy.

MLA, unabsorbed

The third result is MLA in its unabsorbed form, a second attention class beside Week 3's `CausalSelfAttention`. Its latent width d_c and its rotary width d_r are configurable, and d_r must be even. You count the $d_c + d_r$ elements that each token stores per layer, reconstruct every head's content key and value from the latent, and test causality, the shapes and the cache-element arithmetic. The cache holds only the stored elements, so the reconstructed per-head keys and values do not count toward it.

Exercise: Measurements Need Named Boundaries Read on your own

- Analytic cache elements
- Peak allocated memory, synchronized latency
- Backend, device, dtype, shape, warmup, repeats
- Intensity, achieved rates, single-device MFU
- Window-tradeoff run; the supplied trio

▼ What the measurement record keeps apart

The exercise's record reports four kinds of evidence separately, because each comes from a different source.

1. Analytic counts. The stored cache elements of each attention shape follow from the configuration, so they come out the same for everyone.
2. Measurements on your device. The peak allocated memory is read with the memory statistics reset before the timed forwards. The synchronized latency is timed with the device synchronized around the timed forwards. Both depend on your device, the backend that served the shape and your run.
3. What produced each point. The record names the backend you requested and the backend that was selected, or the failure if the request was refused. The backend is the SDPA kernel that actually computed the attention. The record also gives the device, dtype, batch size, head shape, sequence length, warmup and repeats.
4. The roofline. Each point gets its arithmetic intensity, the achieved model FLOP/s, the achieved bandwidth and the single-device MFU, all under one named convention for traffic, precision, timing region and dense or sparse peak. The FLOP count follows the record's dense-total convention. Its attention term counts the whole square of query-key pairs that a masked backend multiplies, because that square is the work the hardware does, so it includes masked pairs that are not useful work. The record gives the FLOPs of the valid pairs beside it.

The window-tradeoff run

The window-tradeoff run is a provided script, `exercises/week-04/window_tradeoff.py`, which you run but do not write. Every backend the protocol measures multiplies the whole masked square, so the window saves no work there. This run shows where it can. It times attention alone in three ways: under SDPA with the window's band, pinned to cuDNN, and under compiled FlexAttention twice, once with a causal block mask and once with a block mask built from your `window_mask_mod`. The windowed block mask lets the kernel skip the tiles outside the window. The run also counts, as arithmetic, the positions an eight-layer stack would keep cached if each local layer evicted what its window no longer reads, for a dense, an all-local and an interleaved stack. Finally it reads the supplied trio, three models that the course staff trained at one budget and that differ only in reach: dense, all-local and interleaved. Their results are reported as held-out perplexity and as mean loss over buckets of positions.

52

To-dos until next week



Muddiest point

Do it right now! (2 min)

Before the tutorial

- Write down one question to discuss
- Think about the reflection questions

Self-study

- Read the reading slides and notes

Deadlines

- Quiz 2: Friday 13 November
- Third exercise: Wednesday 11 November, 22:00

Week 5 turns from what attention costs on a GPU to how a model is trained: optimizers, learning-rate schedules, and numerical stability.

53

Reflection questions

1. Recalculate the H100 SXM ridge using decimal units. Which conclusion changes if a sparse peak is paired with dense attention?
2. A square BF16 GEMM has idealized intensity $2n/(3b)$. Which HBM-transfer and reuse assumptions make that expression optimistic?
3. Under the inclusive SWA convention, what is the maximum receptive field of 24 layers with $w=512$? Which two additional implementation choices are required for work and cache savings?
4. Why does the prepared retained-prefix intervention support a cache-policy result without proving that every softmax head needs a sink?
5. For $n_{kv}=8$, $d_h=128$, $d_c=512$, and $d_r=64$, compare GQA and MLA stored cache elements. Why is attended-token count a different axis?
6. Compare DSA and MSA on selection granularity, sharing, bytes read, index metadata, and kernel shape. Why is 2,048 not a general design constant?

References

References: Hardware and the roofline

- Chowdhery, A., Narang, S., Devlin, J., et al. (2023). PaLM: Scaling Language Modeling with Pathways. Journal of Machine Learning Research, 24(240), 1–113.
- Gholami, A., Yao, Z., Kim, S., Hooper, C., Mahoney, M. W., & Keutzer, K. (2024). AI and the Memory Wall. IEEE Micro, 44(3).
- NVIDIA (2026a). NVIDIA H100 Tensor Core GPU. Official H100 SXM product specifications. Accessed 2026-07-14.
- NVIDIA (2026b). NVIDIA Hopper Architecture In-Depth. Official architecture documentation. Accessed 2026-07-14.
- NVIDIA (2026c). Hopper Tuning Guide, CUDA 13.0. Official tuning documentation. Accessed 2026-07-14.
- Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An Insightful Visual Performance Model for Multicore Architectures. Communications of the ACM, 52(4).

References: FlashAttention

- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. Advances in Neural Information Processing Systems 35 (NeurIPS 2022).
- Dao, T. (2024). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. International Conference on Learning Representations (ICLR 2024).
- PyTorch Contributors (2026). torch.nn.functional.scaled_dot_product_attention. Official stable documentation. Accessed 2026-07-14.
- PyTorch Contributors (2026). FlexAttention and FlashAttention-4: Fast and Flexible. Official PyTorch article, March 2026. Accessed 2026-07-14.
- Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., & Dao, T. (2024). FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. Advances in Neural Information Processing Systems 37 (NeurIPS 2024). DOI: 10.52202/079017-2193.
- Zadouri, T., Hoehnerbach, M., Shah, J., Thakkar, V., & Dao, T. (2026). FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling. Proceedings of Machine Learning and Systems (MLSys 2026).

References: Sliding-window attention and sinks

- Baek, C., Arya, D., Kim, E., et al. (2026). A.X K2 Technical Report. arXiv preprint arXiv:2608.30181.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., et al. (2023). Mistral 7B. arXiv preprint arXiv:2310.06825.
- Microsoft AI (2026). MAI-Thinking-1: Building a Hill-Climbing Machine. Technical report.
- OpenAI (2025). gpt-oss-120b & gpt-oss-20b Model Card. arXiv preprint arXiv:2508.10925.
- Qiu, Z., Wang, Z., Zheng, B., et al. (2025). Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free. Advances in Neural Information Processing Systems 38 (NeurIPS 2025).
- Singh, V., Krauss, L., Jaghouar, S., Sirovatka, M., Goddard, C., et al. (2026). Arcee Trinity Large Technical Report. arXiv preprint arXiv:2602.17004.
- Sun, M., Chen, X., Kolter, J. Z., & Liu, Z. (2024). Massive Activations in Large Language Models. Conference on Language Modeling (COLM 2024).
- Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2024). Efficient Streaming Language Models with Attention Sinks. International Conference on Learning Representations (ICLR 2024).

References: Compression and trainable sparsity

- DeepSeek-AI (2024). DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv preprint arXiv:2405.04434.
- DeepSeek-AI (2025). DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. arXiv preprint arXiv:2512.02556.
- DeepSeek-AI (2026a). DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv preprint arXiv:2606.19348.
- DeepSeek-AI (2026b). DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression. arXiv preprint arXiv:2609.19969.
- Lai, X., Xu, W., Yang, Y., et al. (2026). MiniMax Sparse Attention. arXiv preprint arXiv:2606.13392.
- StepFun Team (2026). Inside Step 5 Preview: Architecture and Training for Agentic Work. StepFun Open Platform blog post, 15 October 2026. Accessed 2026-10-30.
- Verma, S., & Vaidya, N. (2023). Mastering LLM Techniques: Inference Optimization. NVIDIA Technical Blog.
- Yuan, J., Gao, H., Dai, D., et al. (2025). Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. arXiv preprint arXiv:2502.11089.

Image credits

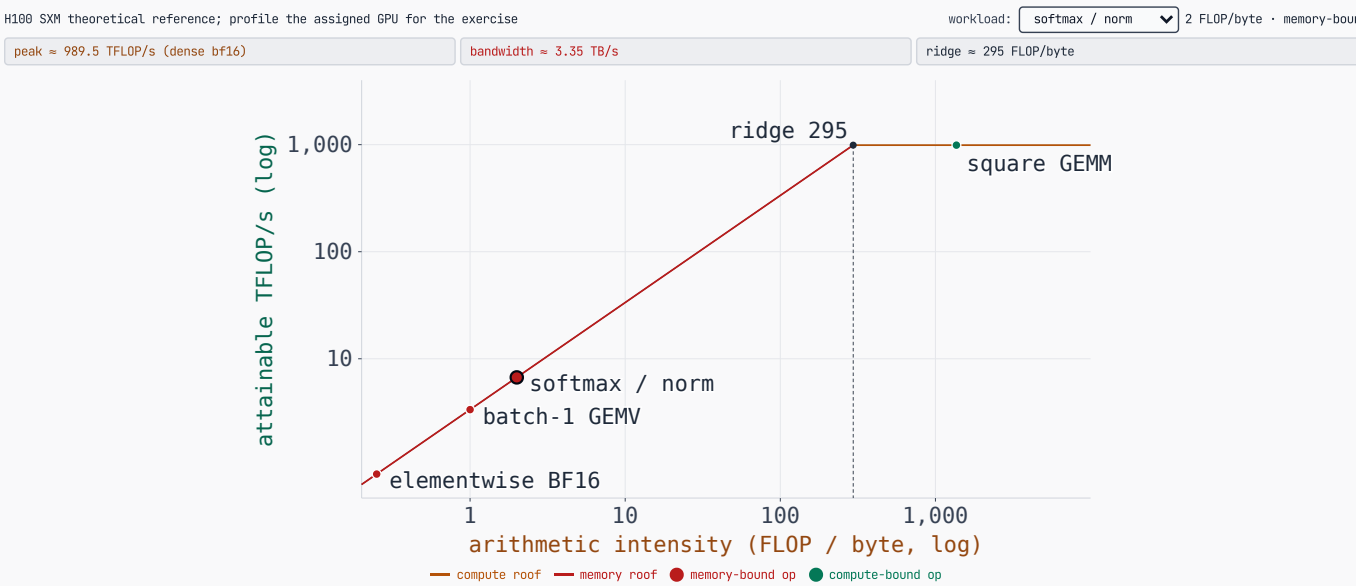
- Sichem, C. van (1603). Steering axle of Stevin's sailing chariot in plan, with two courtiers, one pointing his cane at it. *Diagram van de zeilwagen van Simon Stevin*. Rijksmuseum. CC0.
- Agricola, G. (1556). Early mine wagon on wooden rails, with loose wheels. *De re metallica*. National Library of Poland. Public domain.
- Leupold, J. (1727). Stepped drums, carry wheels and dials of Leibniz's calculating machine. *Theatrum arithmetico-geometricum*, Tab. VIII. Library of Congress. Public domain.
- Descartes, R. (1637). Descartes's microscope on a stand, aimed into a beam of sunlight, with an observer. *La dioptrique*. Library of Congress. Public domain.
- Ramelli, A. (1588). Water wheel driving a ring of piston pumps through a crown gear. *Le diverse et artificiose machine*, Figure LXII. Science History Institute. Public domain.
- Ramelli, A. (1588). Two screw presses side by side, with spare parts on the floor. *Le diverse et artificiose machine*, Figure CLV. Science History Institute. Public domain.
- Zonca, V. (1607). Horse-driven capstan hauling a boat on a wheeled cradle up a ramp over a dam. *Novo teatro di machine et edificii*, p. 58. Library of Congress. Public domain.

Answers

A1 · An H100 SXM needs about 300 FLOPs per byte

attainable throughput

$$P_{\text{attainable}} = \min(P_{\text{peak}}, I \times \beta_{\text{HBM}})$$



The peak is the dense-equivalent BF16 Tensor Core rate: NVIDIA's 2:4-sparse headline divided by two.

Each point's intensity counts useful FLOPs per byte of the modeled BF16 HBM read and write traffic.

The square BF16 GEMM with n = 4,096 has the idealized intensity 2n/(3b = 1,365 FLOP/byte, where b = 2 bytes per element.

The roofline caps the throughput an operation can reach. Left of the ridge, bandwidth limits it; right of it, compute can. The ridge is peak compute divided by bandwidth.

The operation placements are illustrative. Elementwise assumes one read and one write per element, batch-1 GEMV reads the weights once per token, softmax/norm streams its values, and square GEMM reuses its data heavily. The exact placement depends on shape, dtype, cache traffic and fusion.

▼ The ridge point separates the two roofs

An H100 SXM in BF16 needs about 300 FLOPs for every byte it reads from or writes to HBM to keep its arithmetic units busy. Two reference numbers set this scale: the dense BF16 Tensor Core peak of 989.5 TFLOP/s, and the HBM bandwidth of 3.35 TB/s (NVIDIA, 2026a).

From a bound on time to a bound on throughput

The roofline model turns the lower bound on time into an upper bound on throughput (Williams et al., 2009). An operation that performs F FLOPs in time T runs at F/T FLOP/s. Write β_{HBM}, or β for short, for the HBM bandwidth, B for the HBM bytes, and $I = F/B$ for the intensity.

Claim (upper bound). Given the lower bound $T \geq \max(F/P_{\text{peak}}, B/\beta)$ and $F > 0$, the operation's throughput satisfies

$$\frac{F}{T} \leq P_{\text{attainable}} = \min(P_{\text{peak}},\ I \times \beta_{\text{HBM}}).$$

The key step is that for positive numbers $1/\max(x, y) = \min(1/x, 1/y)$. Dividing F by the bound on T therefore gives $\min(P_{\text{peak}}, F\beta/B)$, which is $\min(P_{\text{peak}}, I\beta)$. This attainable throughput is a ceiling: an operation can run slower, but under the time bound it cannot run faster.

Two roofs meet at the ridge

Plotted against intensity, the bound has two roofs: a bandwidth roof $I\beta$ that rises with intensity, and a flat compute roof at P_{peak}. They meet where $I\beta = P_{\text{peak}}$, at the intensity

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{\beta},$$

which Williams et al. call the **ridge point** (Williams et al., 2009). Below the ridge, bandwidth sets the attainable throughput, and the workload is memory-bound. Above it, compute can set it, and the workload is compute-bound.

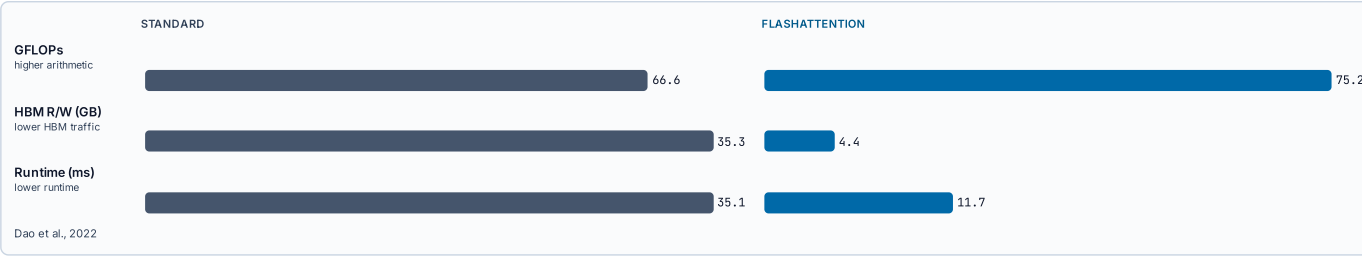
Reading the roofline chart

The chart draws this roofline for the H100 SXM reference on logarithmic axes, intensity against attainable throughput, so the bandwidth roof is a straight sloped line. Four markers place four operations: an elementwise operation in BF16, a matrix-vector product at batch size 1 (batch-1 GEMV), a softmax or normalization, and a square matrix multiplication (square GEMM). Their color shows which side of the ridge each one falls on, and

hovering over a marker shows the traffic model behind its position. A different traffic model moves a marker, and a different device peak or bandwidth moves the ridge. The chart applies analytic assumptions to one device's theoretical peaks, so the speed of a particular shape, kernel or system still has to be measured.

Back to the question

A2 · Fewer bytes beat extra arithmetic



- HBM traffic: 35.3 → 4.4 GB (Dao et al., 2022)
- Runtime: 35.1 → 11.7 ms
- Recomputing costs some FLOPs and saves far more traffic

▼ Recomputation traded a little arithmetic for much less traffic

The time went into moving bytes. Figure 2 of Dao et al.'s final NeurIPS paper compares standard attention with FlashAttention in one setup: the attention of GPT-2 medium on an A100 GPU, forward and backward pass together. The sequence length is $S = 1{,}024$ (the paper's N) and the head width $D_h = 64$, with 16 heads, batch size 64, a key-padding mask and no dropout (Dao et al., 2022).

Row	Standard	FlashAttention	Change
Arithmetic	66.6 GFLOPs	75.2 GFLOPs	$75.2/66.6 = 1.129$, 12.9% more
HBM reads and writes	35.3 GB	4.4 GB	$35.3/4.4 = 8.02$, 87.5% less
Runtime	35.1 ms	11.7 ms	$35.1/11.7 = 3.0$, 66.7% less

The extra arithmetic comes from recomputing blocks in the backward pass (Dao et al., 2022). In return, HBM traffic falls by a factor of 8.02 and runtime by a factor of 3.0.

The lower bound from the Memory Wall tab shows how more arithmetic can take less time:

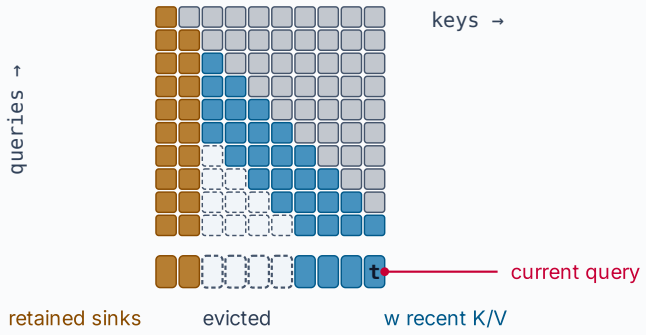
$$T \geq \max\left(\frac{F}{P_{\text{peak}}},\ \frac{B}{\beta}\right).$$

The time follows the resource that binds. Recomputation raises the FLOPs F a little and lowers the bytes B a lot, which pays when the avoided traffic costs more time than the added arithmetic, as it did here. An operation count alone does not tell you which resource binds, so optimize the one your measurements show binding for the shape and backend you run.

Figure 2 describes this one setup. Its ratios do not carry over to other shapes, devices, pass directions, implementations or backends.

Back to the question

A3 · Evicting the first tokens breaks a rolling window



ILLUSTRATIVE Adapted from Xiao et al., 2024

The schematic follows Xiao et al.'s Figure 1(d). StreamingLLM keeps the key-value pairs of the initial positions beside the recent window. Here the retained entries are ordinary sequence positions. The paper's optional learnable sink token, trainec in its §3.3, is not drawn.

- Window alone: perplexity 5,158 (Xiao et al., 2024)
- Keep 4 first tokens: 5.40
- Llama-2-13B, 65K-token PG19 book

▼ Keeping the first tokens restores the rolling window

If some heads park their spare weight on the first token, the softmax hypothesis predicts trouble for a rolling cache. Such a cache keeps the keys and values of the most recent tokens and evicts its oldest entry each time a new token arrives, so once the text is longer than the cache, the first tokens are gone and the heads that used them would lose their sink. Suppose the cache keeps the last 1,024 tokens. Once the first ones are evicted, does perplexity barely change, or does it get much worse?

It gets much worse. Xiao et al. measure this in Table 1 of their StreamingLLM paper, with Llama-2-13B on the first book of the PG19 test set, a corpus of public-domain books, 65K tokens long. A cache configuration `x+y` keeps x initial tokens beside the y most recent ones (Xiao et al., 2024):

Configuration	What the cache keeps	Perplexity
`0+1024`	the 1,024 most recent tokens, the window alone	5,158.07
`4+1020`	the 4 initial tokens and the 1,020 most recent	5.40

Their method, StreamingLLM, keeps these sinks beside the recent window, and in their setups four initial tokens suffice to restore the model's performance (Xiao et al., 2024). Four is the count for the models and texts they report, not a constant.

The attention maps are an observation, and the retention result is an intervention. Neither says how common sinks are or what causes them.

The widget draws the policy on ten positions, with queries as rows and keys as columns. Every query reads two retained initial positions and its own window of four, and the keys in between are evicted. The strip below is the cache at the current query t, the rightmost cell. Each retained cell is a real position with its own key and value. The sizes are chosen for the drawing, and the drawing does not implement eviction.

What the retained positions cost

The retained positions are real cache entries. With w recent and n_{sink} retained positions, $T_{\text{visible}} = w + n_{\text{sink}}$, and the sinks add $2 \cdot L \cdot H_{kv} \cdot D_h \cdot n_{\text{sink}} \cdot b$ bytes per sequence to the window's cache, with the symbols of the sliding-window slides.

Positions in StreamingLLM and in the exercise

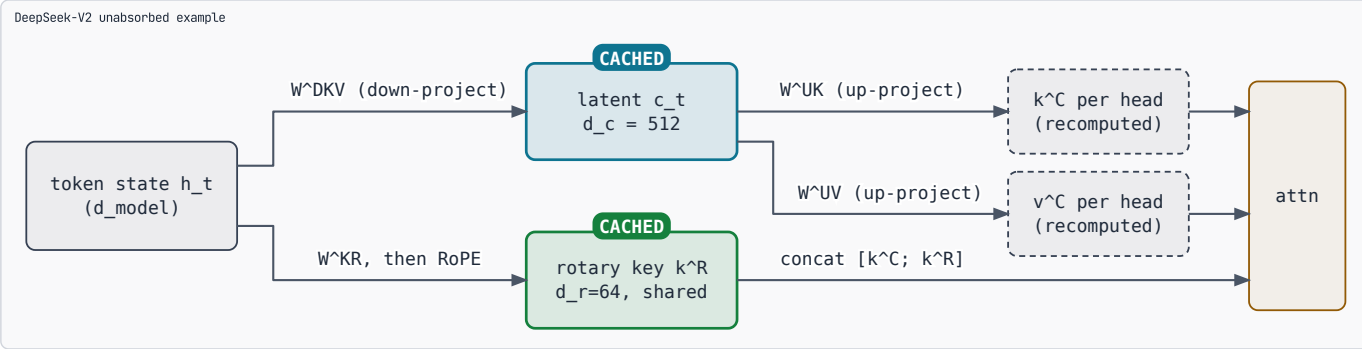
StreamingLLM computes relative distances and adds positional information from each token's position within the cache, not its position in the original text (Xiao et al., 2024). The exercise does not re-index, so positions stay absolute. It also chooses the retained positions differently. The course's trace, `assets/week-04/sink-trace.json`, names them from attention measured on the course's sink model. Its `sink_positions` field lists `[0, 731, 734, 825]`, and only the first is an initial position.

Back to the question

A4 · MLA caches 576 numbers, not 32,768

stored elements per token

$$2(128)(128) = 32768$$
$$512 + 64 = 576$$
$$1 - 576/32768 = 98.24\%$$



- 98.24% fewer than the MHA comparator (DeepSeek-AI, 2024)
- Counted elements, not measured memory

▼ 576 stored elements per token against 32,768

Full MHA in this configuration caches 32,768 numbers per token per layer. How many does MLA cache, with its latent and one rotary key? About 600: it caches 576.

Assumptions

- **A1. DeepSeek-V2's dimensions.** The model has $n_h = 128$ heads of width $d_h = 128$, a latent of width $d_c = 512$ and a rotary key of width $d_r = 64$ (DeepSeek-AI, 2024).
- **A2. The comparator.** Full MHA with the same heads, each caching its own key and value, so $n_{kv} = n_h = 128$.
- **A3. Elements, then bytes.** Stored elements per token per layer, for one sequence. BF16 takes two bytes per element, and $1\,\mathrm{KiB} = 2^{10}$ bytes.

Claim. Under A1 and A2, MLA stores 576 elements per token per layer against the comparator's 32,768, which is 98.24% fewer. Under A3 that is 1.125 KiB against 64 KiB. The two formulas of the previous note give the counts:

$$\begin{aligned} 2\,n_{kv}\,d_h &= 2 \cdot 128 \cdot 128 = 32{,}768 && \text{A1, A2} \\ d_c + d_r &= 512 + 64 = 576 && \text{A1.} \end{aligned}$$

The reduction is one minus their ratio:

$$1 - \frac{576}{32{,}768} = 1 - 0.017578125 = 0.982421875,$$

so MLA stores 98.24% fewer elements than the MHA comparator. In bytes,

$$\begin{aligned} 32{,}768 \cdot 2 &= 65{,}536 \text{ bytes} = 64 \text{ KiB} && \text{A3} \\ 576 \cdot 2 &= 1{,}152 \text{ bytes} = 1.125 \text{ KiB} && \text{A3.} \end{aligned}$$

These numbers are analytic counts of the stored tensors, as with the window's cache. Memory measured on a device would differ, because the rebuilt keys and values take memory of their own while attention runs, and the traffic of a decode step is a separate count again. DeepSeek-V2 reports its own figure, a KV cache 93.3% smaller than that of its baseline model, which is a different comparison from the MHA comparator here (DeepSeek-AI, 2024).

Reading the diagram

From the token state h_t on the left (labeled `d_model`), W^{DKV} leads to the latent c_t in cyan, and W^{KR} followed by RoPE to the shared rotary key k^R in green, both marked CACHED. From the latent, W^{UK} and W^{UV} lead to the per-head content keys and values in dashed boxes labeled

"recomputed", rebuilt at each use and never stored. Each head's content key is concatenated with the shared rotary key, $[k^C; k^R]$. Only the two cached boxes count toward the 576.

Back to the question